

Chương 1. Giới thiệu

1.1 Đồ thị :

1.1.1 Định nghĩa đồ thị vô hướng (*undirected graph*):

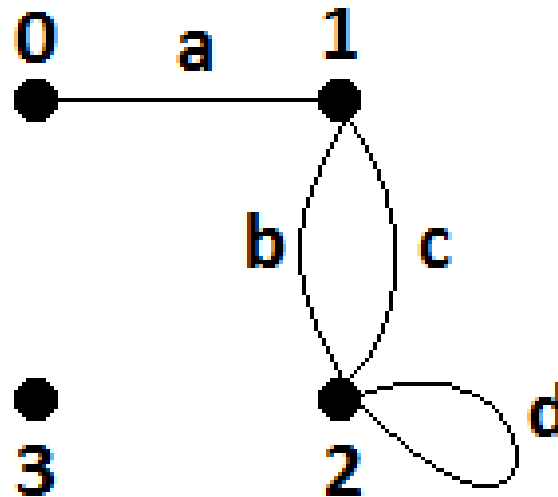
- Đồ thị G là một bộ (V, E) , với V là tập khác $\emptyset$, hữu hạn các phần tử được gọi là các *đỉnh* (*vertex*), E là tập các phần tử được gọi là các *cạnh* (*edge*). Mỗi phần tử $e \in E$ liên kết với duy nhất một cặp đỉnh $v, w \in V$ và được ký hiệu $e = (v, w)$, $v, w \in V$.
- $e = (v, w)$ được gọi là cạnh *liên thuộc* với hai đỉnh v và w (e is called *incident with* the vertices u and w), và v và w là các đỉnh *kề nhau* (*the two vertices v and w are adjacent*).

- $e_1 = (v, w)$ và $e_2 = (v, w)$, e_1 và e_2 được gọi là 2 cạnh song song (*multiple edges*).
 - $e = (v, v)$ được gọi là một khuyên hay vòng (*loop*).
 - $d(v)$, gọi là *bậc* của đỉnh v (*the degree of the vertex v*), là số cạnh kề với v . Nếu đỉnh v có vòng thì vòng được tính là 2.
 - Đỉnh có bậc bằng 0 được gọi là đỉnh cô lập.
 - Đỉnh có bậc bằng 1 được gọi là đỉnh treo.
- Lưu ý :** $e = (v, w) = (w, v)$ (Không kể thứ tự).

Ví dụ : Cho $G=(V, E)$, $V=\{0, 1, 2, 3\}$, $E=\{ a=(0, 1), b=(1, 2), c=(1, 2), d=(2, 2)\}$.

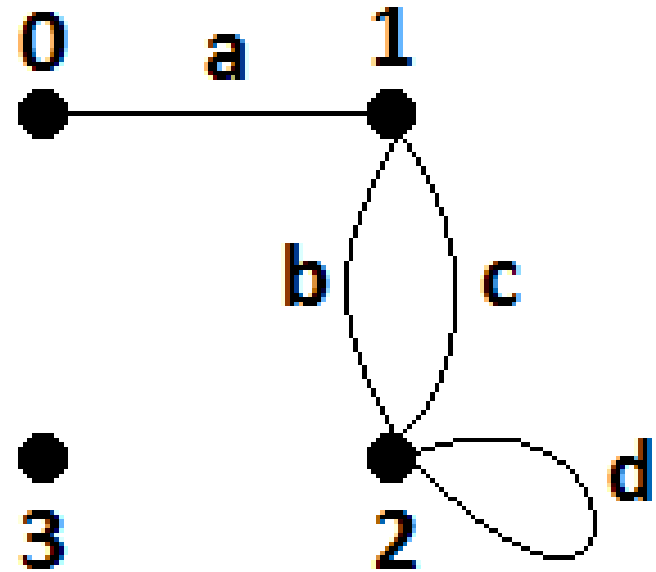
Đồ thị G được biểu diễn bằng hình ảnh như sau:

- Cạnh a là *cạnh liên thuộc* với đỉnh 0 và đỉnh 1, đỉnh 0 *kề* đỉnh 1.
- Cạnh b song song cạnh c , $b \parallel c$.
- Cạnh d là vòng.



- Bậc của các đỉnh :

v	$d(v)$
0	1
1	
2	
3	



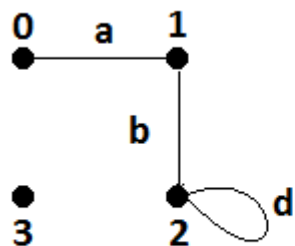
- Đỉnh 0 là *đỉnh treo*.
- Đỉnh 3 là *đỉnh cô lập*.

1.1.2 Định nghĩa đồ thị đơn giản (simple graph) :

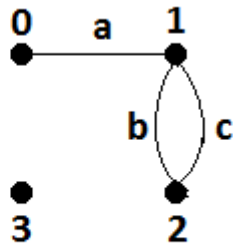
Cho G là đồ thị. Nếu G không có cạnh song song và không có vòng thì được gọi là đồ thị đơn giản.

Ví dụ :

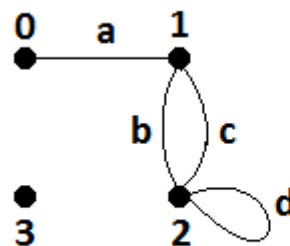
- Các đồ thị (G1, G2, G3) không đơn giản :



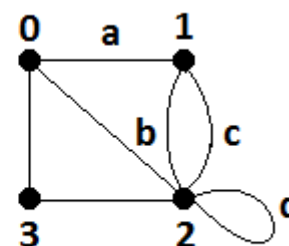
G1



G2



G3



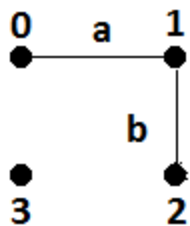
G4

Multi graph

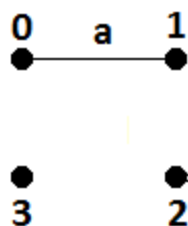
Pseudo graph

Adjacent graph

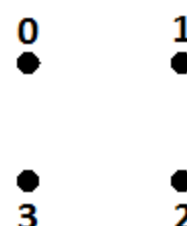
- Các đồ thị đơn giản :



G1



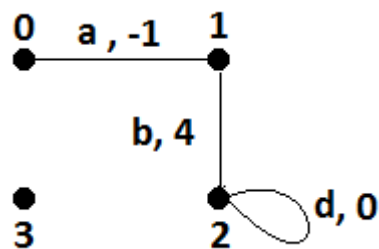
G2



G3

Null graph

1.1.3 Định nghĩa trọng số (*weight*): Cho $G = (V, E)$ là đồ thị. Mỗi cạnh $e \in E$ ta gán cho e một giá trị số thực được gọi là trọng số của e . Khi đó đồ thị G được gọi là đồ thị có trọng số.



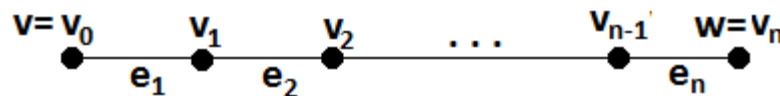
- Cạnh a có trọng số là -1 ,
- Cạnh b có trọng số là 4 ,
- Cạnh d có trọng số là 0 .

1.2 Đường đi (*path*):

1.2.1 Định nghĩa đường đi : Cho $G = (V, E)$ là đồ thị.

Cho $v, w \in V$ ta định nghĩa đường đi nối đỉnh v và w , ký hiệu $P(v, w)$, như sau:

$P(v, w)$ là dãy $\{ e_i = (v_{i-1}, v_i) : i=1, \dots, n, v=v_0, v_n=w \}$ (1)

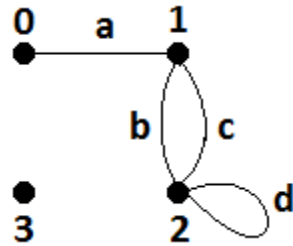


➤ **Chú ý :** Ta cũng có thể biểu diễn $P(v, w)$ như sau

$$P(v, w) = (v_0, e_1, v_1, e_2, v_2, \dots, v_{n-1}, e_n, v_n) \quad (2)$$

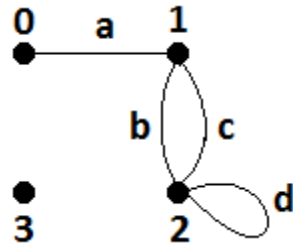
- $|P(v, w)| = n$, là chiều dài đường đi.
- Một trường hợp đặc biệt của đường đi là đường đi chỉ gồm một đỉnh v (không có cạnh) khi đó đường đi được ký hiệu là (v) , có chiều dài bằng không.
- Nếu không tồn tại dãy e_i ở (1) ta nói không có đường đi giữa v và w , ta qui ước $P(v, w) = \emptyset$.
- Hai đường đi P và Q là khác nhau nếu chúng có ít nhất một cạnh là khác nhau.

Ví dụ :



- $P1 = P(0, 1) = \{a = (0, 1)\}$, hay $P1 = P(0, 1) = (0, a, 1)$,
 - $P2 = P(0, 1) = \{a = (0, 1), a = (1, 0), a = (0, 1)\}$, hay
 $P2 = P(0, 1) = (0, a, 1, a, 0, a, 1)$,
 - $P3 = P(0, 1) = \{a = (0, 1), b = (1, 2), b = (2, 1)\}$, hay
 $P3 = P(0, 1) = (0, a, 1, b, 2, b, 1)$,
 - $P4 = P(0, 1) = \{a = (0, 1), b = (1, 2), c = (2, 1)\}$, hay
 $P4 = P(0, 1) = (0, a, 1, b, 2, c, 1)$,
- Các đường đi $P1, P2, P3, P4$ là khác nhau.

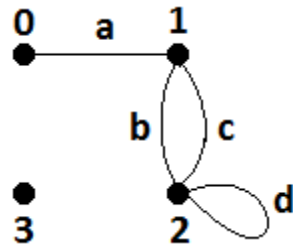
Ví dụ :



- $P_5 = P(0,2) = \{a=(0,1), b=(1,2), c=(2,1), c=(1,2), d=(2,2)\}$, hay
 $P_5 = P(0,2) = (0,a,1,b,2,c,1,c,2,d,2),$
- $P_6 = P(0,3) = \emptyset.$

1.2.2 Định nghĩa chu trình (*circuit*): Cho $G = (V, E)$ là đồ thị. Một chu trình **đỉnh v** là $P(v, v)$.

Ví dụ :



$P(0,0)=\{a=(0,1), b=(1,2), c=(2,1), a=(1,0)\}$, hay

$P(0,0)=(0,a,1,b,2,c,1,a,0)$,

1.2.3 Định nghĩa đường đi , chu trình đơn giản (*simple path, circuit*):

Cho $G = (V, E)$ là đồ thị.

- Một đường đi gọi là *đơn giản* ,nếu không có *cạnh* nào xuất hiện quá một lần.
 - Một chu trình đỉnh v gọi là *đơn giản* ,nếu không có *cạnh* nào xuất hiện quá một lần.
- Ta chỉ cần kiểm tra công thức $P(v,w)$ ở dạng (1) xem có cạnh nào xuất hiện nhiều hơn một lần không.

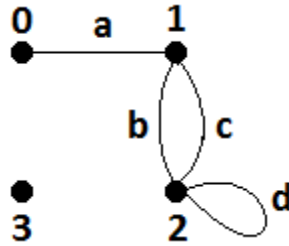
1.2.4 Định nghĩa đường đi , chu trình sơ cấp

(elementary path, circuit):

Cho $G = (V, E)$ là đồ thị.

- Một đường đi gọi là *sơ cấp* ,nếu không có *đỉnh* nào xuất hiện quá một lần.
 - Một chu trình đỉnh v gọi là *sơ cấp* ,nếu không có *đỉnh* nào xuất hiện quá một lần (đỉnh đầu và cuối được xem là một đỉnh).
- Ta chỉ cần kiểm tra từng đỉnh v trong công thức $P(v,w)$ ở dạng (2). Đếm v , xem số lần ≥ 2 không.
 - Đối với chu trình thì đỉnh đầu và đỉnh cuối được tính là 1.

Ví dụ :



$P(0,0)=\{a=(0,1), b=(1,2), c=(2,1), a=(1,0)\}$, dạng (1)

$P(0,0)=(0,a,1,b,2,c,1,a,0)$, dạng (2).

Chu trình trên không đơn giản vì cạnh a xuất hiện 2 lần, không sơ cấp, vì đỉnh 1 xuất hiện 2 lần ở dạng (2).

$P(1,1) = \{b=(1,2), c=(2,1)\}$ hay

$P(1,1) = \{1,b,2,c,1\}$

Chu trình vừa đơn vừa sơ cấp.

1.3 Liên thông (Connectivity) :

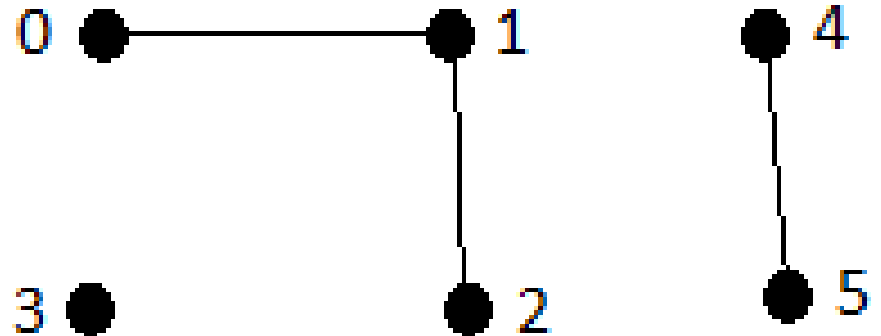
1.3.1 Định nghĩa quan hệ $\approx$: Cho $G = (V, E)$ là đồ thị. $v, w \in V$,

$$v \approx w \Leftrightarrow (v = w) \vee (P(v, w) \neq \emptyset).$$

VD : Cho đồ thị G

Ta có :

- $0 \approx 0$ vì $0=0$,
 $0 \approx 1$ vì có đường đi nối 0 và 1,
 $0 \approx 2$ vì có đường đi nối 0 và 2,
- $1 \approx 1, 1 \approx 0, 1 \approx 2$,
- $2 \approx 2, 2 \approx 0, 2 \approx 1$,
- $3 \approx 3$,
- $4 \approx 4, 4 \approx 5$,
- $5 \approx 5, 5 \approx 4$.
- 1 không có quan hệ ≈ 3 , vì $P(1,3) = \emptyset$
- 1 không có quan hệ ≈ 4 .



1.3.2 Định nghĩa thành phần liên thông (connected component) :

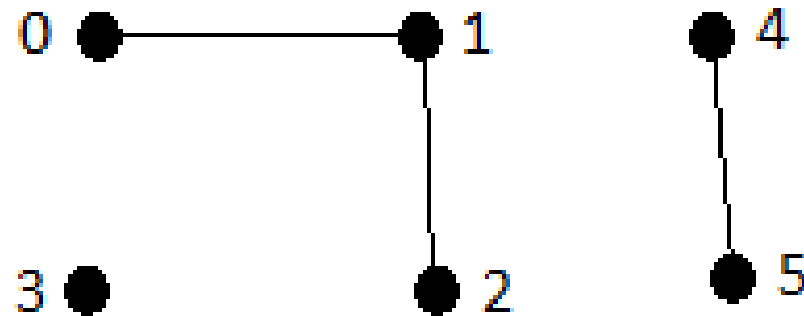
Cho $G = (V, E)$ là đồ thị. Ta định nghĩa $[v] = \{ w \in V : v \approx w \}$. $[v]$ được gọi là thành phần liên thông *chứa* v .

Vd : Từ ví dụ ở Slide trước cho ta

$$[0]=[1]=[2]=\{0,1,2\},$$

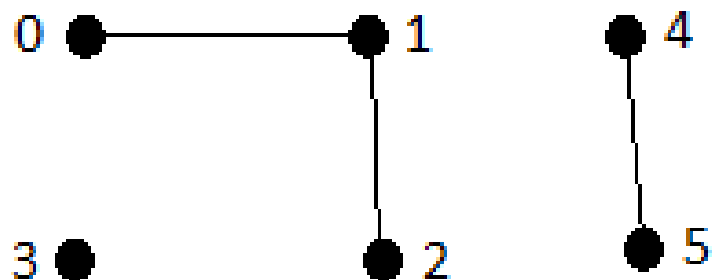
$$[3]=\{3\},$$

$$[4]=[5]=\{4,5\}.$$



1.3.3 Định nghĩa liên thông: Cho $G = (V, E)$ là đồ thị. G được nói là liên thông nếu số thành phần liên thông của G là 1.

Vd :



Đồ thị ở trên không liên thông vì có 3 (>1) thành phần liên thông ($\{0,1,2\}$, $\{3\}$ và $\{4,5\}$).

1.4 Đồ thị có hướng (directed graph):

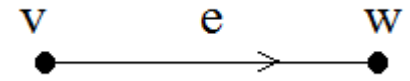
1.4.1 Định nghĩa đồ thị có hướng : Đồ thị có hướng

$G=(V, E)$ là đồ thị được định nghĩa ở **1.1.1** với mỗi cạnh $e=(v,w)$ có phân biệt thứ tự v và w , tức là $(v,w) \neq (w,v)$.

- Lưu ý : Khi nói “cho đồ thị G . . .” thì ta sẽ hiểu đồ thị G theo định nghĩa ở Các Slide trước (còn gọi là đồ thị vô hướng). Khi nói “cho đồ thị G có hướng” thì G được định nghĩa ở 1.4.1.
- Những gì ta học ở các Slide trước cũng được áp dụng cho đồ thị có hướng.

➤ $e=(v, w) \in E$ được nói là cạnh / cung hướng từ v đến w , *hướng ra* từ v , *hướng vào* w .

➤ Nếu $e=(v, w) \in E$, ta nói w *kề với* v ,
 v là đỉnh đầu (the initial vertex), w là đỉnh cuối (the end vertex).



➤ Cho $e_1=(v, w) \in E$ và $e_2=(v, w) \in E$. Khi đó e_1 và e_2 được nói là 2 cung song song cùng hướng (multiple directed edges).

➤ Cho $e_1=(v, w) \in E$ và $e_2=(w, v) \in E$. Khi đó e_1 và e_2 được nói là 2 cung song song ngược hướng.

- Cho $e=(v, v)$, e được gọi là một vòng, *vừa hướng ra từ v vừa hướng vào v* . Nếu một đỉnh có 2 vòng thì ta xem chúng là song song cùng hướng.
- $d^-(v)$: số cạnh hướng vào v , *nửa bậc trong (in-degree)* của đỉnh v .
- $d^+(v)$: số cạnh hướng ra từ v , *nửa bậc ngoài (out-degree)* của đỉnh v .
- Đồ thị có hướng được nói là đồ thị đơn giản nếu không có vòng, không có cạnh song song cùng hướng.

1.4.2 Định nghĩa đường đi : Cho $G=(V, E)$ có hướng.
Một đường đi trong G từ v đến w , ký hiệu $P(v, w)$,
được định nghĩa :

$P(v, w)$ là dãy $\{ e_i=(v_{i-1}, v_i) : i=1,2,\dots,n, v=v_0, w=v_n \}$. (1)

- $|P(v, w)| = n$ là chiều dài đường đi.
- Một trường hợp đặc biệt của đường đi là đường đi chỉ gồm một đỉnh v (không có cạnh) khi đó đường đi được ký hiệu là (v) , có chiều dài bằng không.
- Nếu không tồn tại dãy e_i ở (1) ta nói không đường đi từ v đến w , ta qui ước $P(v, w) = \emptyset$.

Chú ý : $P(v, w) \neq P(w, v)$.

1.4.3 Định nghĩa chu trình : Cho $G=(V, E)$ có hướng.
Một chu trình (có hướng) đỉnh v là $P(v, v)$.

- Đường đi đơn, đường đi sơ cấp, chu trình đơn, chu trình sơ cấp được định nghĩa như đồ thị không hướng.

1.4.4 Định nghĩa $\approx$: Cho $G=(V, E)$ có hướng. $v, w \in V$.

$$v \approx w \Leftrightarrow (v=w) \vee (P(v, w) \neq \emptyset \wedge P(w, v) \neq \emptyset).$$

1.4.5 Định nghĩa lớp tương đương : $v \in V$.

$$[v] = \{ w \in V : w \approx v \}.$$

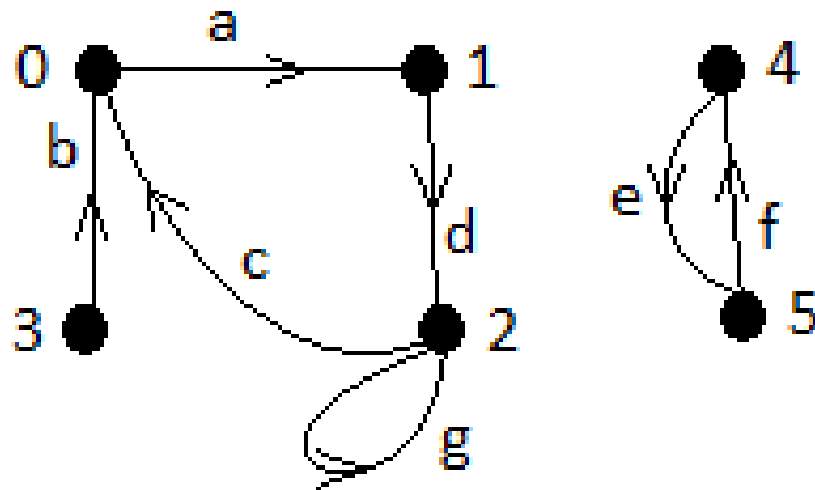
$[v]$ được gọi là *thành phần liên thông mạnh (strong component)* chứa v .

1.4.6 Định nghĩa liên thông mạnh : Nếu G (có hướng) có đúng 1 thành phần liên thông mạnh thì ta nói G liên thông mạnh.

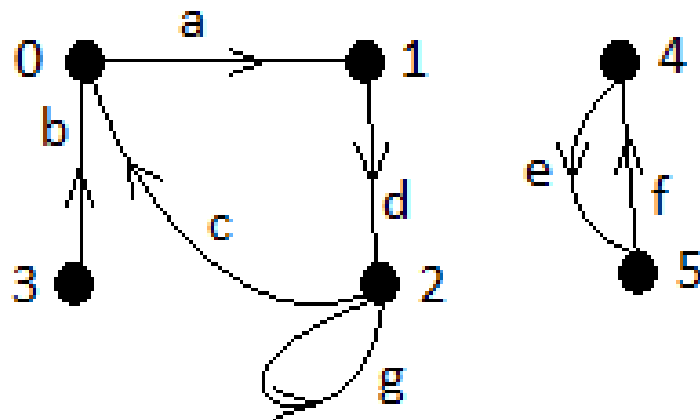
VD : Cho đồ thị có hướng G :

- Đường đi $P(3,2) = \{b=(3,0), a=(0,1), d=(1,2)\}$, nhưng không có đường đi $P(2,3)$.
- Bậc

v	$d^-(v)$	$d^+(v)$
0	2	1
1	1	1
2	2	2
3	0	1
4	1	1
5	1	1



VD : Cho đồ thị có hướng G :



- Quan hệ $\approx$:

$0 \approx 0$,

$0 \approx 1$, vì có $P(0,1)$ và $P(1,0)$,

0 không quan hệ $\approx$ với 3, vì có $P(3,0)$ nhưng không có $P(0,3)$.

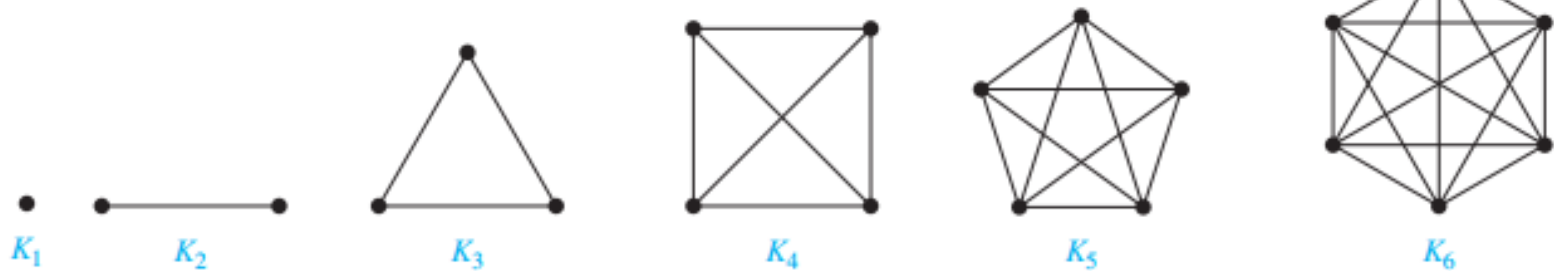
- Các thành phần liên thông mạnh :

$[0]=[1]=[2]=\{0,1,2\}$, $[3]=\{3\}$, $[4]=[5]=\{4, 5\}$.

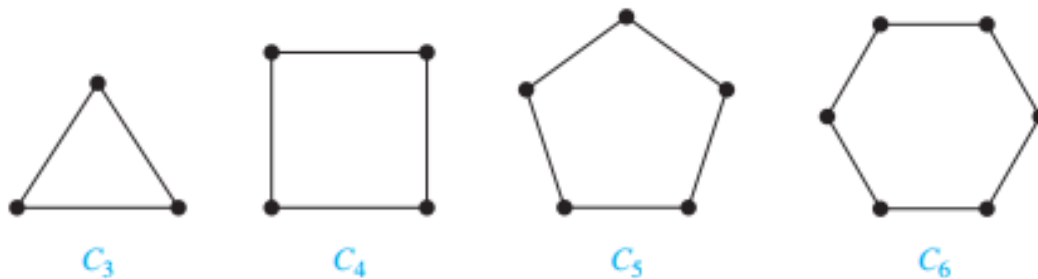
Đồ thị đã cho không liên thông mạnh vì có 3 (>1) thành phần liên thông mạnh ($\{0,1,2\}$, $\{3\}$ và $\{4,5\}$).

Some simple graphs :

complete graphs on n vertices K_n

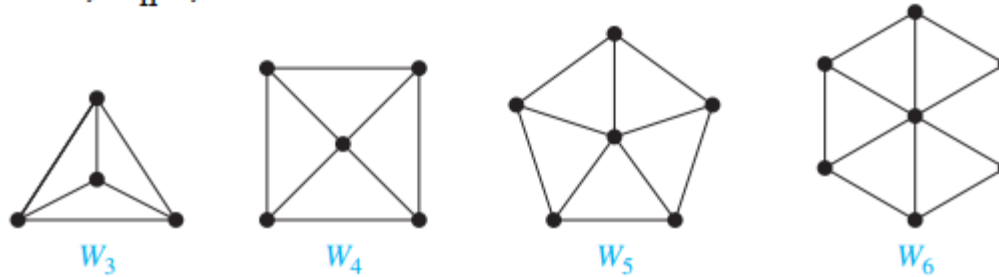


Cycles, C_n , $n \geq 3$



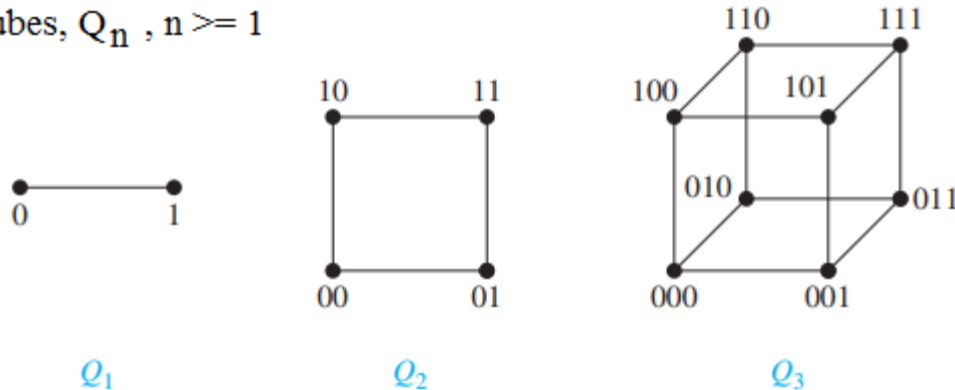
Some simple graphs :

Wheels, W_n , $n \geq 3$



The Wheels W_3 , W_4 , W_5 , and W_6 .

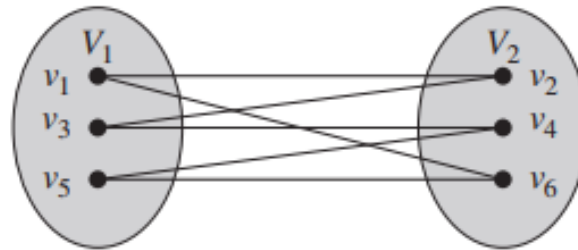
Cubes, Q_n , $n \geq 1$



The n -cube Q_n , $n = 1, 2, 3$.

Some simple graphs :

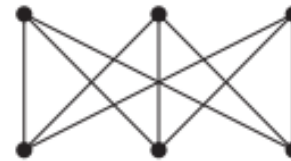
Bipartite graph



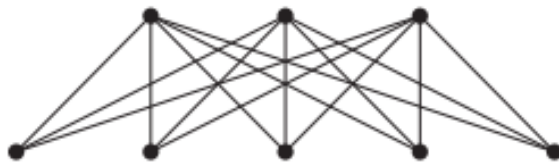
Complete bipartite graph



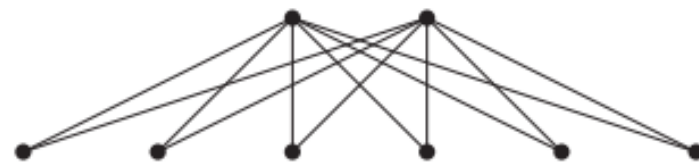
$K_{2,3}$



$K_{3,3}$



$K_{3,5}$



$K_{2,6}$

Tài liệu tham khảo:

1. Discrete Mathematics , Richard Johnsonbaugh
2. Algorithms, Thomas h. Cormen
3. Toán Rời Rạc Nâng Cao, Trần Ngọc Danh, ĐHQG TP HCM
4. Lý Thuyết Đồ Thị, Đặng Trường Sơn, Lê văn Vinh, ĐHSP Kỹ Thuật TP HCM

Chương 2. Đường đi ngắn nhất

2.1 Biểu diễn đồ thị bằng ma trận :

2.1.1 Biểu diễn bằng ma trận kề :

2.1.1.1 Định nghĩa: Cho $G=(V, E)$, tập các đỉnh $V=\{0, \dots, n-1\}$. Ma trận kề **A** của **G** là **ma trận vuông**, cấp n có **a_{ij} được định nghĩa:**

a_{ij} số cạnh liên thuộc với đỉnh i và đỉnh j .

i chỉ số hàng, j chỉ số cột.

Ví dụ : Ma trận kề A được lập như bảng sau :

A =

	0	1	2	3
0	0	1	0	0
1	1	0	2	0
2	0	2	1	0
3	0	0	0	0

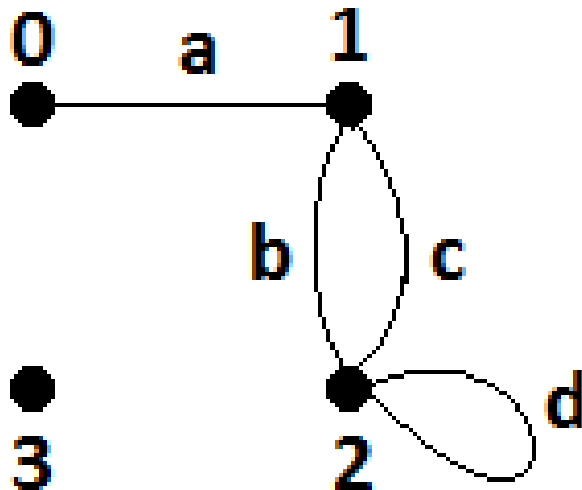
- a_{ij} số cạnh liên thuộc với đỉnh i và đỉnh j.
- i chỉ số hàng, j chỉ số cột.

$$a_{12} = 2$$

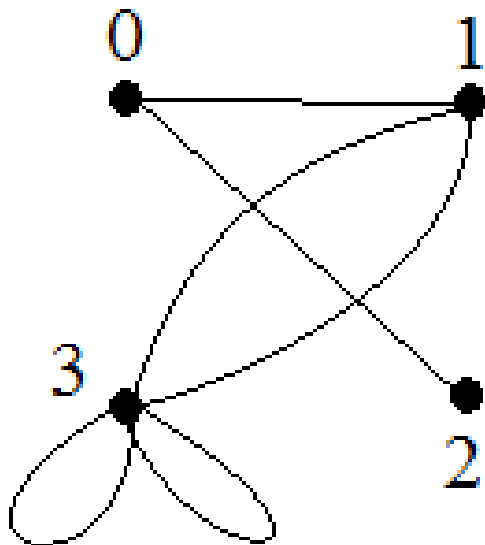
(đỉnh 1 và 2 có 2 cạnh liên thuộc với b và c)

$$a_{22} = 1$$

(đỉnh 2 có 1 cạnh liên thuộc với d)



a_{ij} số cạnh liên thuộc với đỉnh i và đỉnh j



	0	1	2	3
0				
1				
2				
3				

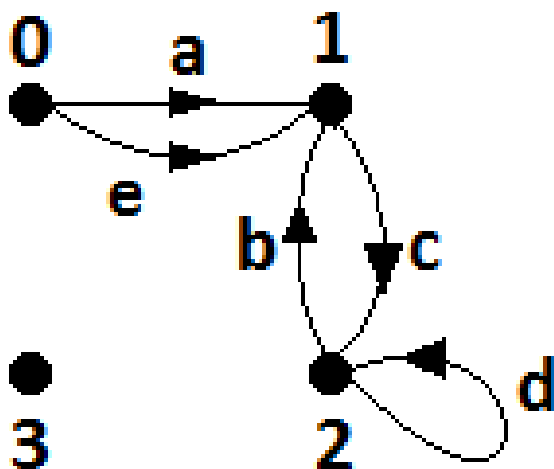
- Ma trận kề của **G có hướng** :

a_{ij} là số cạnh hướng từ đỉnh i đến đỉnh j .

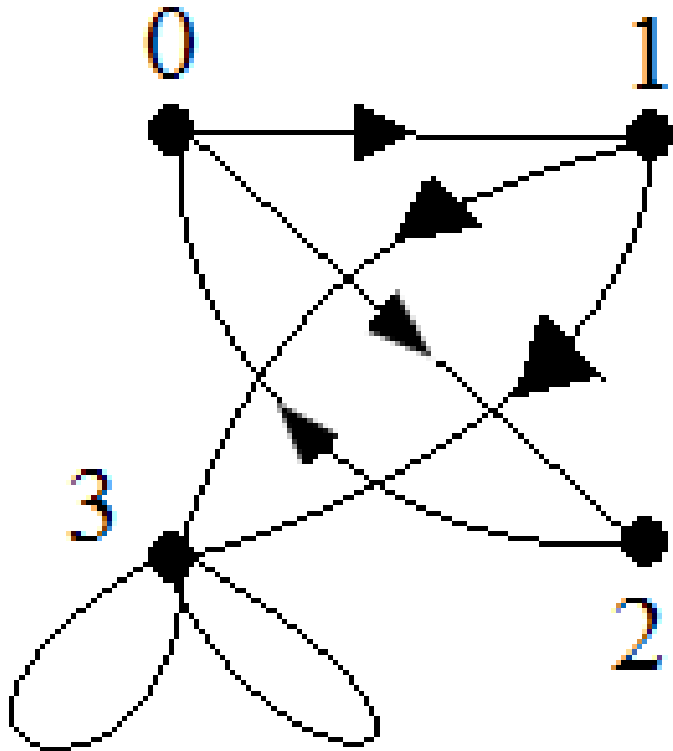
	0	1	2	3
0	0	2	0	0
1	0	0	1	0
2	0	1	1	0
3	0	0	0	0

$a_{01} = 2$
(có 2 cạnh kề a và e hướng từ đỉnh 0 đến đỉnh 1)

$a_{22} = 1$
(đỉnh 2 có 1 cạnh d hướng đến chính nó)



a_{ij} là số cạnh hướng từ đỉnh i đến đỉnh j



	0	1	2	3
0				
1				
2				
3				

2.1.2 Biểu diễn bằng ma trận liên thuộc :

2.1.2.1 Định nghĩa : Cho $G=(V, E)$, có $V=\{ 0, 1,...,n-1\}$ và $E =\{e_i : i=0,... , m-1\}$. **Ma trận liên thuộc A** là ma trận có a_{ij} được định nghĩa:

$a_{ij} = 1$ nếu e_j *liên thuộc* với đỉnh i và $a_{ij}=0$ nếu ngược lại.

■ Nếu G có hướng thì

$a_{ij} = 1$, nếu e_j *hướng ra từ đỉnh i* và $a_{ij}= -1$ nếu e_j *hướng vào đỉnh i* .

→ Vòng ở đỉnh i được xem là *hướng ra từ đỉnh i* .

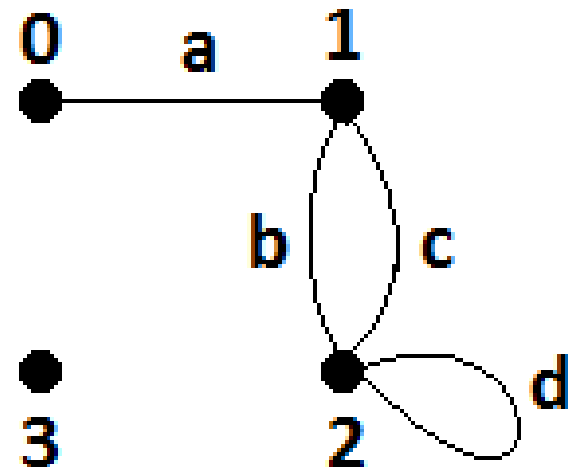
→ $a_{ij} = 0$, nếu e_j không liên kết với đỉnh i .

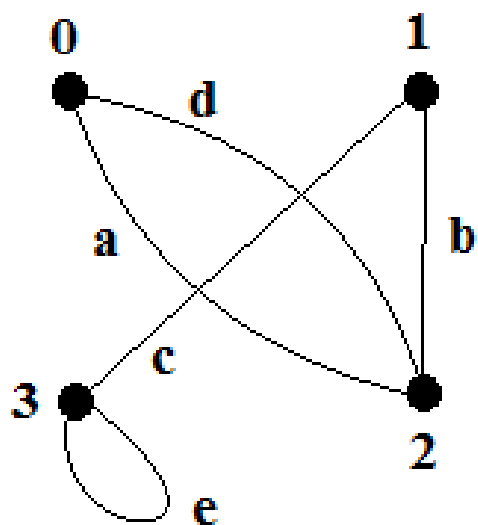
Ví dụ : Ma trận liên kết A được lập như bảng sau :

A =

	e_0	e_1	e_2	e_3
	a	b	c	d
0	1	0	0	0
1	1	1	1	0
2	0	1	1	1
3	0	0	0	0

$a_{1c} = 1$
(cạnh c liên thuộc với 1)





	e_0	e_1	e_2	e_3	e_4
	a	b	c	d	e
0					
1					
2					
3					

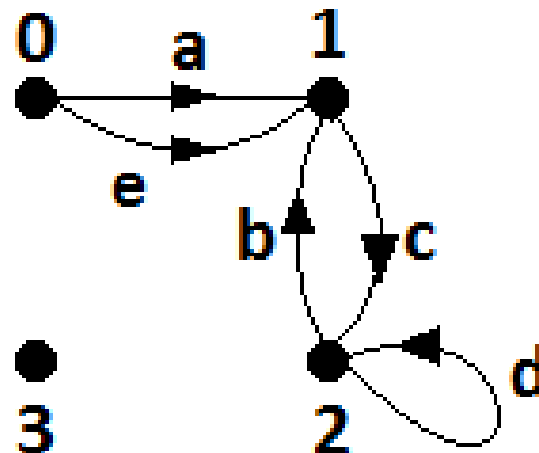
$a_{ij} = 1$ nếu e_j liên thuộc với đỉnh i
 và $a_{ij} = 0$ nếu ngược lại

- Ma trận liên thuộc của G có hướng :

$a_{ij} = 1$, nếu e_j hướng ra từ đỉnh i và $a_{ij} = -1$ nếu e_j hướng vào đỉnh i

	a	b	c	d	e
0	1	0	0	0	1
1	-1	-1	1	0	-1
2	0	1	-1	1	0
3	0	0	0	0	0

$a_{1e} = -1$
(cạnh hướng đỉnh 1)



2.1.3 Biểu diễn bằng ma trận trọng số :

2.1.3.1 Định nghĩa :

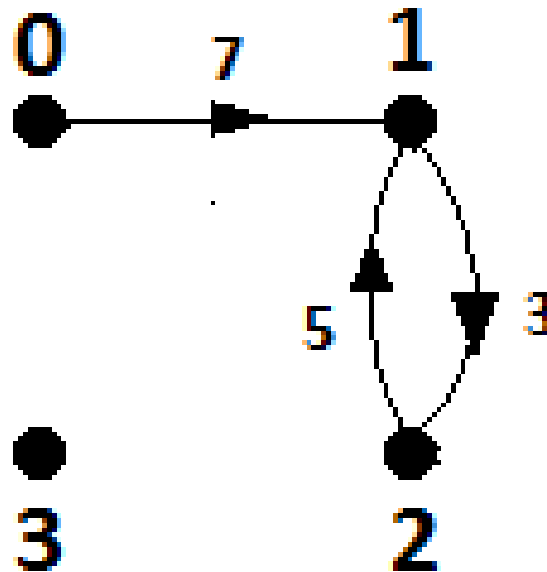
- Đồ thị $G=(V, E)$ có hướng, $V = \{0, 1, 2, \dots, n-1\}$, có trọng số, **không cạnh song song cùng hướng**, **không vòng**. Ma trận trọng số W được định nghĩa :

$$- W = (w_{ij}) = \begin{cases} 0 & \text{nếu } i=j, \\ \text{trọng số của cạnh có hướng } (i, j) \in E & \\ \infty & \text{nếu } i \neq j \text{ và } (i, j) \notin E. \end{cases}$$

- Ma trận trọng số của G có hướng :

	0	1	2	3
0	0	7	∞	∞
1	∞	0	3	∞
2	∞	5	0	∞
3	∞	∞	∞	0

$a_{13} = \infty$
(không có cạnh hướng từ đỉnh 1 đến 3)

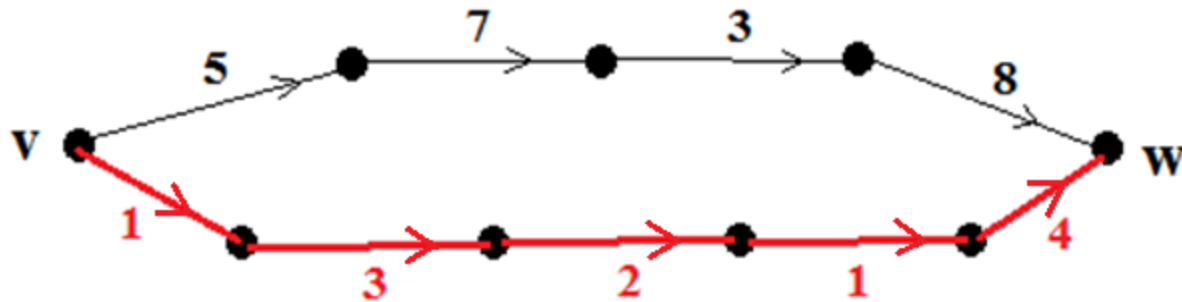


2.2 Đường đi ngắn nhất :

2.2.1 Định nghĩa:

Cho $G=(V, E)$, có trọng số , có hướng.

- **Trọng số** của một đường đi từ đỉnh v đến w là tổng trọng số các cạnh của đường đi đó.
- Đường đi ngắn nhất giữa 2 đỉnh v, w là đường đi trọng số bé nhất.



Qui tắc tính với ∞ :

1) Với mọi $a \in \mathbb{R}$, $a < \infty \Rightarrow a + \infty = \infty$

2) $\infty + \infty = \infty$

3) $\infty = \infty$ (phép so sánh)

4) Với mọi $a \in \mathbb{R}$ không phải là ∞ thì $a < \infty$ (phép so sánh)

➤ Trong cài đặt chương trình, ∞ có thể được thay thế bởi bất kỳ hiệu nào miễn là ký hiệu đó thỏa 4 tính chất trên.

2.3 Thuật toán Dijkstra:

2.3.1 Yêu cầu :

- Đồ thị $G=(V, E)$ có hướng, có trọng số, không cạnh song song cùng hướng , không vòng.
- $W = (w_{ij})$, $w_{ij} \geq 0$ hoặc $w_{ij} = \infty$.
- Thuật toán tìm đường đi ngắn nhất từ đỉnh s đến tất cả các đỉnh .
- Giả sử có đường đi từ s đến tất cả các đỉnh.

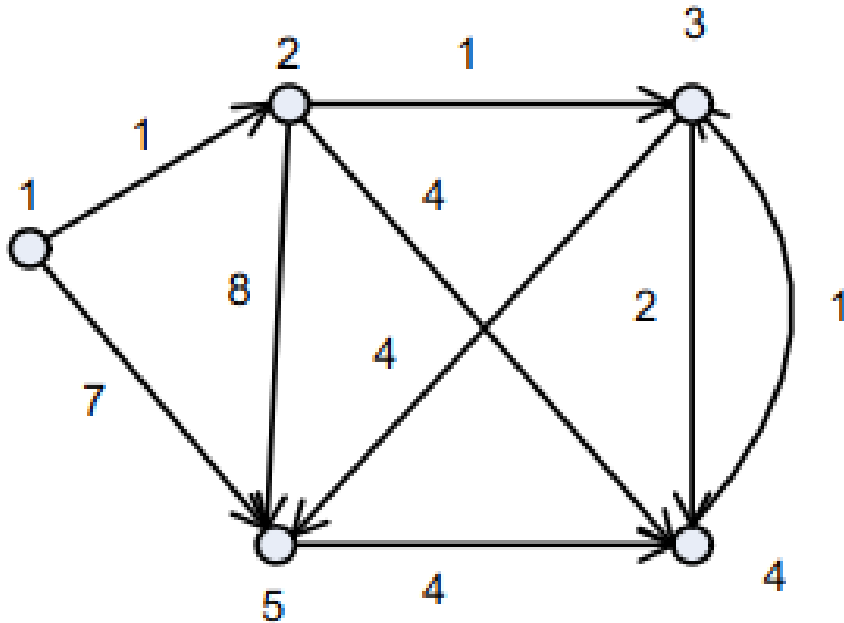
2.3.2 Thuật toán :

```
void Dijkstra
{
  for ( $v \in V$ )
  {
     $d[v] = w[s,v]$ ;
     $Truoc[v] = s$ ;
  }

   $d[s] = 0$ ;
   $T = V \setminus \{s\}$ ;
```

```
while ( $T \neq \emptyset$ )
{
  Tìm  $u \in T$  sao cho  $d[u] = \min \{ d[z] : z \in T \}$ 
   $T = T \setminus \{u\}$ ;
  for ( $v \in T$ ) do
    if ( $d[v] > d[u] + w[u,v]$ ) then
    {
       $d[v] = d[u] + w[u,v]$ ;  $Truoc[v] = u$ ;
    }
  } /* Kết thúc while ( $T \neq \emptyset$ ) */
} /* Kết thúc void Dijkstra */
```

$s = 1$



	1	2	3	4	5
1	∞	1	∞	∞	7
2	∞	∞	1	4	8
3	∞	∞	∞	2	4
4	∞	∞	1	∞	∞
5	∞	∞	∞	4	∞

T	d[2], Truoc[2]	d[3], Truoc[3]	d[4], Truoc[4]	d[5], Truoc[5]
2, 3, 4, 5	1, 1	∞ , 1	∞ , 1	7, 1
3, 4, 5		2, 2	5, 2	7, 1
4, 5			4, 3	6, 3
5				6, 3
$\emptyset$	1, 1	2, 2	4, 3	6, 3

2.4 Thuật toán Bellman-Ford :

Thuật toán tìm đường đi ngắn nhất từ s đến các đỉnh.

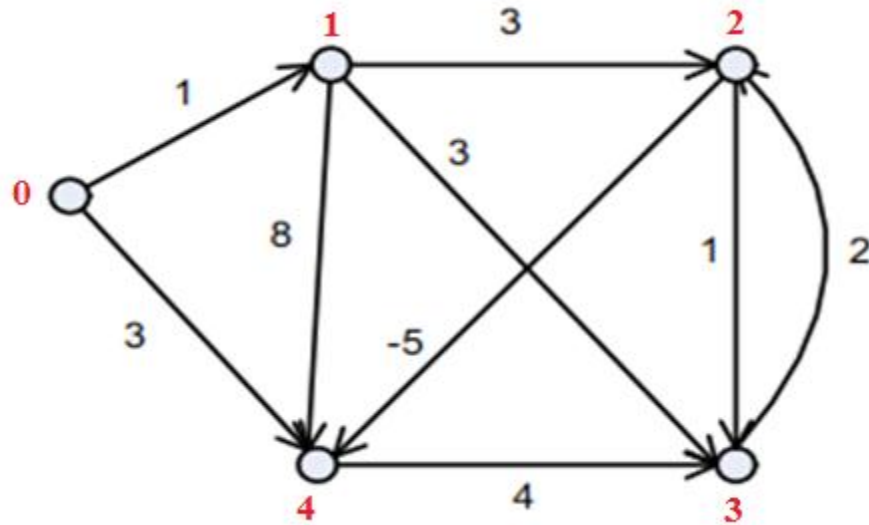
2.4.1 Yêu cầu:

- Đồ thị $G=(V, E)$ có hướng, có trọng số (ma trận W).
- Không có cạnh song song cùng hướng , không có vòng.

2.4.2 Thuật toán :

```
void Ford_Bellman()
{
    for (v ∈ V)
    {
        d[v] = w[s,v];
        Truoc[v] = s;
    }
    d[s] = 0;
```

```
    for (k = 1; k < n-1; k++) // n : số đỉnh
        for (v ∈ V \ {s})
            for (u ∈ V)
                if (d[v] > d[u] + w[u,v])
                {
                    d[v] = d[u] + w[u,v];
                    Truoc[v] = u;
                }
} /* Kết thúc Ford_Bellman()
```



$S = 0$, $v = 4, 3, 2, 1$, $u = 0, 1, 2, 3, 4$

k	d[1], Truoc[1]	d[2], Truoc[2]	d[3], Truoc[3]	d[4], Truoc[4]
	1, 0	∞ , 0	∞ , 0	3, 0
1	1, 0	4, 1	4, 1	3, 0
2	1, 0	4, 1	3, 4	-1, 2
3	1, 0	4, 1	3, 4	-1, 2
	1, 0	4, 1	3, 4	-1, 2

2.5 Thuật toán Floyd-Warshall :

2.5.1 Yêu cầu:

- Đồ thị $G=(V, E)$ có hướng, có trọng số. Không cạnh song song cùng hướng , không vòng, **không chu trình âm**.
- n : số đỉnh.
- $D^{(k)} = (d_{ij}^{(k)})$. $D^{(n)}=(d_{ij}^{(n)})$ là kết quả của đường đi ngắn nhất từ đỉnh i đến đỉnh j .

$$\text{- } P = (p_{ij}), p_{ij} = \begin{cases} -1, & \text{nếu } i=j \text{ hay } w_{ij} = \infty \\ i, & \text{nếu } i \neq j \text{ và } w_{ij} < \infty \end{cases}$$

2.5.2 Thuật toán :

Bước 1 :

$$D^{(0)} = W, P^{(0)} = P$$

Bước 2 :

```
for (k = 1 ; k <= n ; k++) {  
    for (i=1; i<=n; i++) {  
        for (j=1; j<=n; j++){  
            If ( $d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$  )  
            {  $d_{ij}^{(k)} = d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$   
               $p_{ij}^{(k)} = p_{kj}^{(k-1)}$  ;  
            }  
            else {  $d_{ij}^{(k)} = d_{ij}^{(k-1)}$  ;  $p_{ij}^{(k)} = p_{ij}^{(k-1)}$  }  
        } End For j  
    } End For i  
} End For k
```

Thuật toán in đường đi từ i đến j.

Print-Path(i, j)

{

 If (i==j) printf(i);

 Else If ($P_{ij} == -1$) printf(“Khong co duong di i, j.”)

 Else {

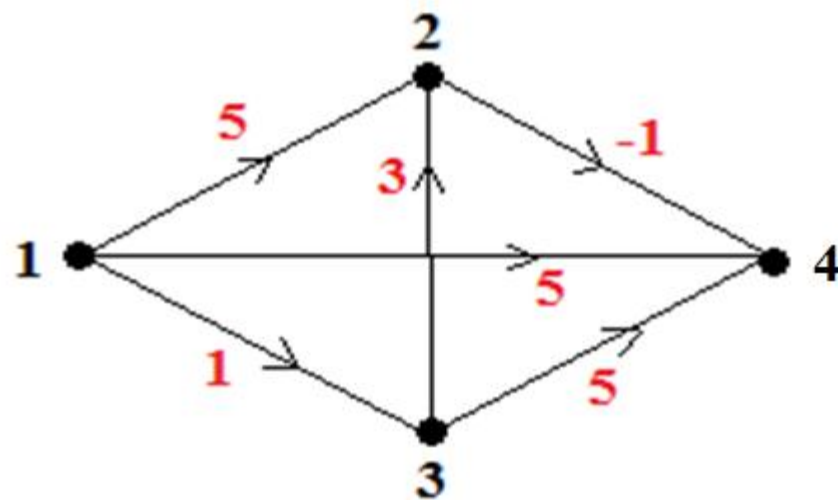
Print-Path(i, P_{ij});

 printf(j);

 }

}

Ví dụ :



k=**1**, 2, 3, 4.

$D^{(0)}$:

	1	2	3	4
1	0	5	1	5
2	vc	0	vc	-1
3	vc	3	0	5
4	vc	vc	vc	0

$D^{(1)}$:

	1	2	3	4
1				
2				
3				
4				

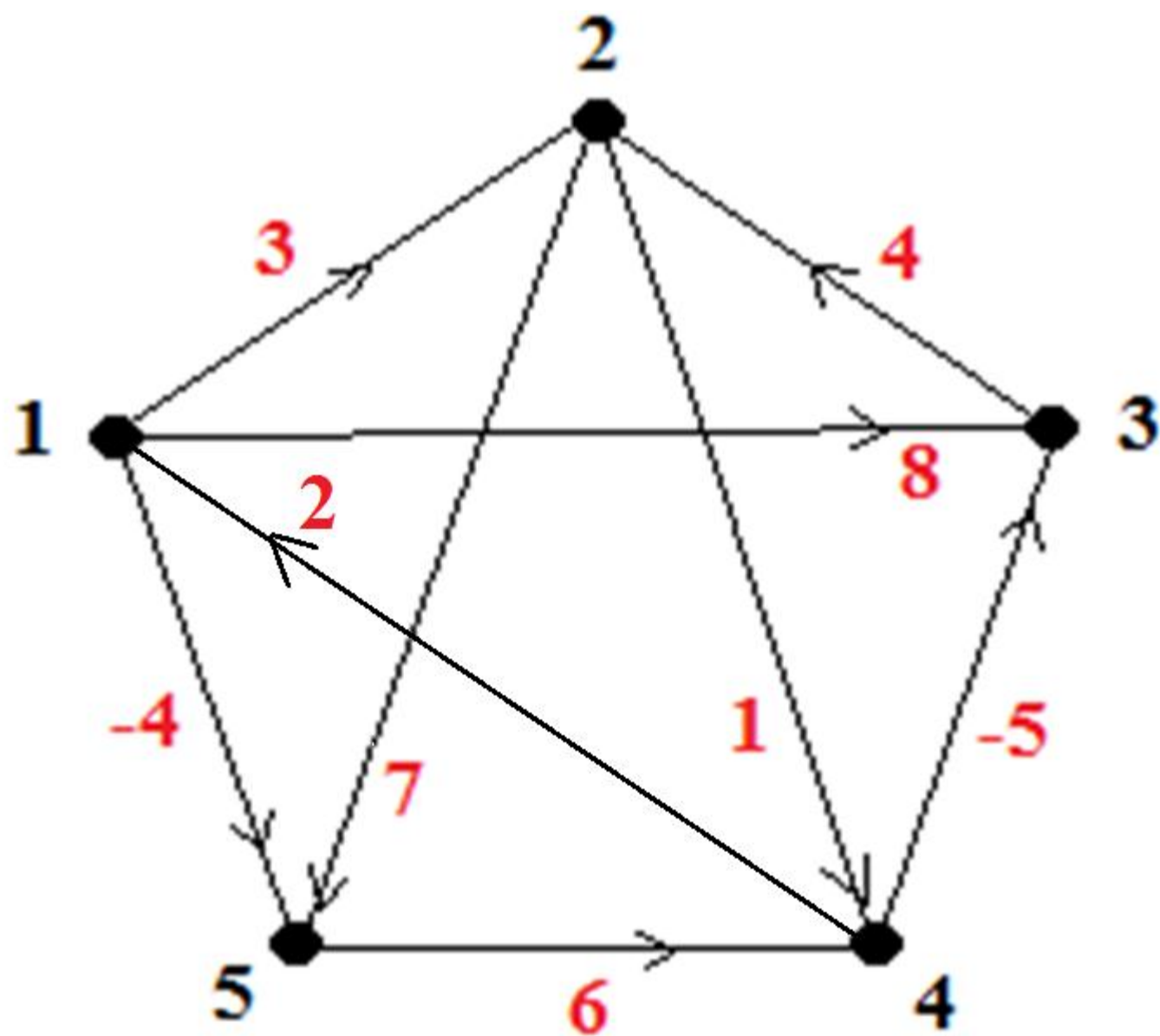
$P^{(0)}$:

	1	2	3	4
1	-1	1	1	1
2	-1	-1	-1	2
3	-1	3	-1	3
4	-1	-1	-1	-1

$P^{(1)}$:

	1	2	3	4
1				
2				
3				
4				

Bài tập :



Bài tập :

$D^{(0)} = ?$, $P^{(0)} = ?$, $D^{(5)} = ?$, $P^{(5)} = ?$, **Print-Path(3, 4) = ?**.

$D^{(4)} =$

0	3	-1	4	-4
3	0	-4	1	-1
7	4	0	5	3
2	-1	-5	0	-2
8	5	1	6	0

$P^{(4)} =$

-1	1	4	2	1
4	-1	4	2	1
4	3	-1	2	1
4	3	4	-1	1
4	3	4	5	-1

Bài giải:

Print-Path(3, 4) = ?.

$D^{(5)} =$

0	1	-3	2	-4
3	0	-4	1	-1
7	4	0	5	3
2	-1	-5	0	-2
8	5	1	6	0

$P^{(5)} =$

-1	3	4	5	1
4	-1	4	2	1
4	3	-1	2	1
4	3	4	-1	1
4	3	4	5	-1

Tài liệu tham khảo:

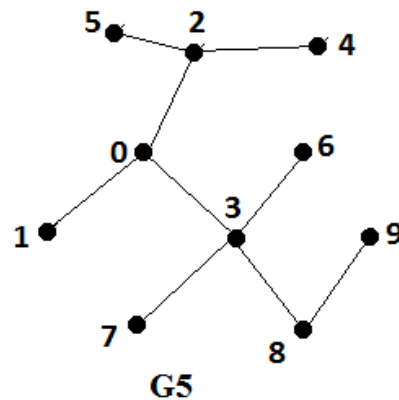
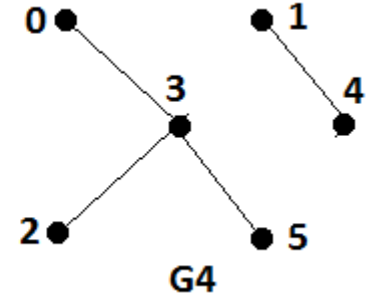
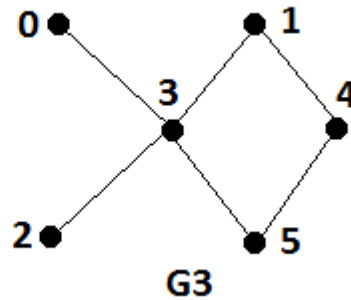
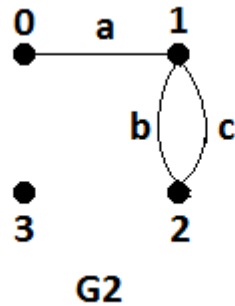
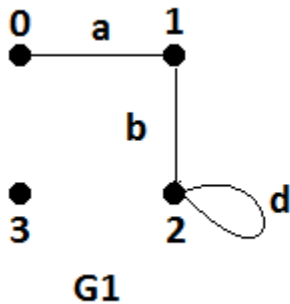
1. Discrete Mathematics , Richard Johnsonbaugh
2. Algorithms, Thomas h. Cormen
3. Toán Rời Rạc Nâng Cao, Trần Ngọc Danh,
ĐHQG TP HCM
4. Lý Thuyết Đồ Thị, Đặng Trường Sơn, Lê văn
Vinh, ĐHSP Kỹ Thuật TP HCM

Chương 3. Cây

3.1 Cây :

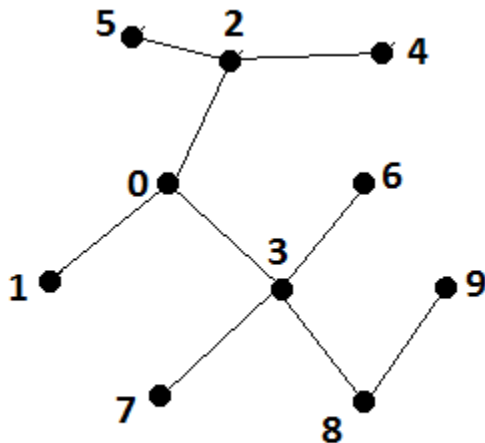
3.1.1 Định nghĩa cây :

Cây T là một **đồ thị đơn giản** thỏa : Nếu v và w là hai đỉnh trong T , thì có một **đường đi sơ cấp** duy nhất nối v và w.

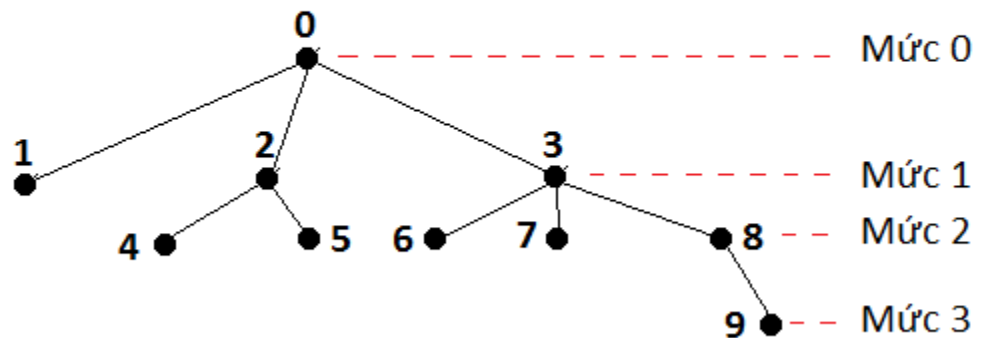


3.1.2 Định nghĩa cây có gốc:

- Cây T có gốc là cây mà trong T một đỉnh v nào đó được chọn làm gốc (duy nhất).
- Độ dài (số cạnh) **đường đi sơ cấp** từ gốc đến đỉnh v được gọi mức của v. Gốc có mức 0.
- Mức cao nhất được gọi là **chiều cao của cây**.



H1



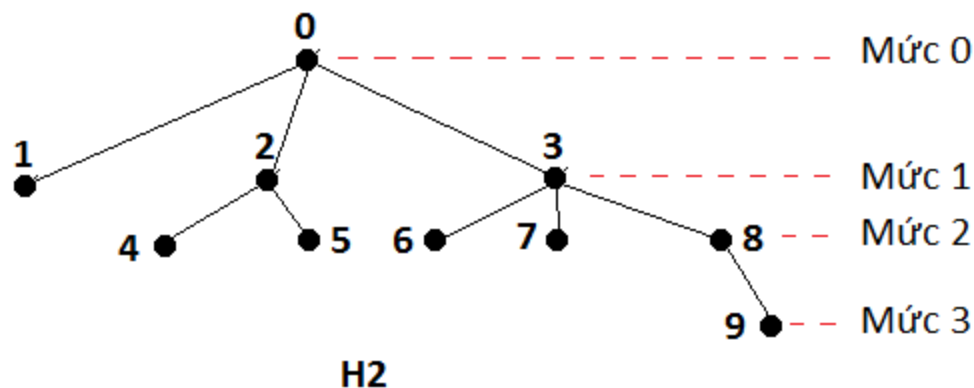
H2

3.1.3 Định nghĩa các thuật ngữ:

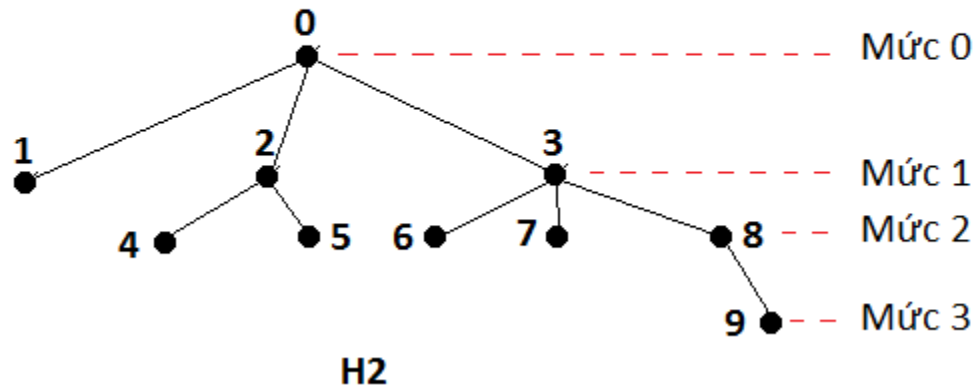
Cho T có gốc v_0 . Giả sử x, y , và z các đỉnh trong T , và $(v_0, v_1, \dots, v_{n-1}, v_n)$ đường sơ cấp trong T :

- v_{n-1} đỉnh (nút) cha của v_n .
- $v_0, v_1, \dots, v_{n-1}$ tổ tiên của v_n .
- v_n là con của v_{n-1} .
- Nếu x là tổ tiên của y , thì y là con cháu của x .
- Nếu x và y là các con của z , thì x và y anh em.
- Nếu x không có con, x là lá.
- Nếu x không là lá, x là đỉnh trong.

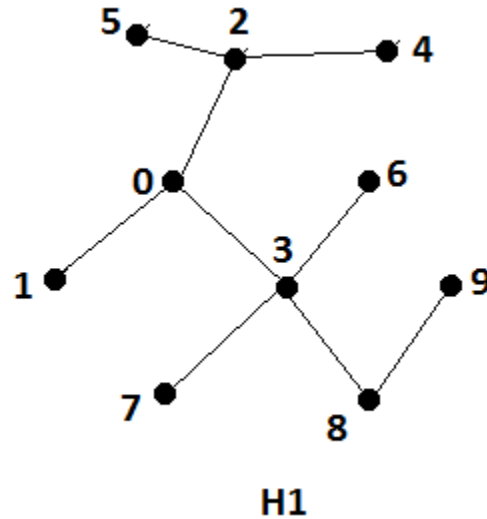
(0, 3, 8)



- **Cây con của T có gốc x** là đồ thị với tập đỉnh V và tập cạnh E , trong đó
 - + V gồm x và tất cả các con cháu của x ,
 - + $E = \{ e : e \text{ là một cạnh trên đường đi sơ cấp từ } x \text{ tới một đỉnh nào đó thuộc } V \}$.

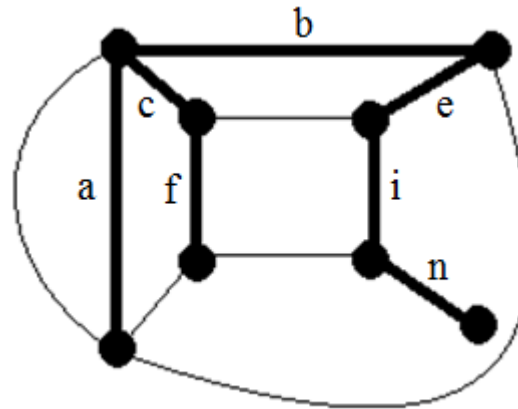
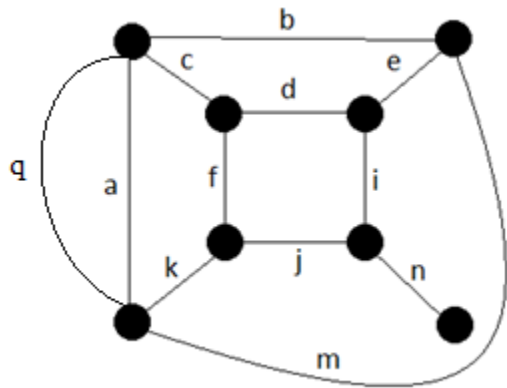


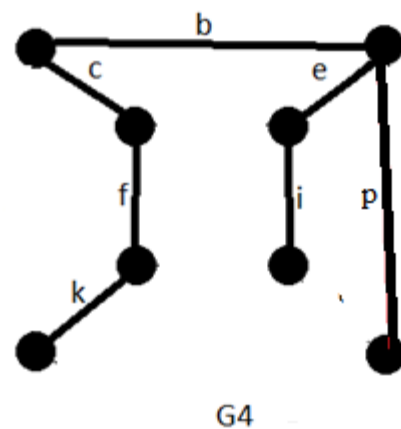
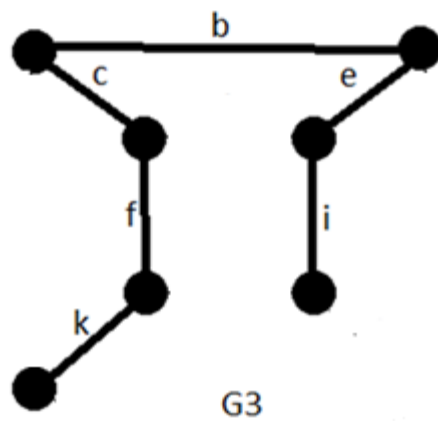
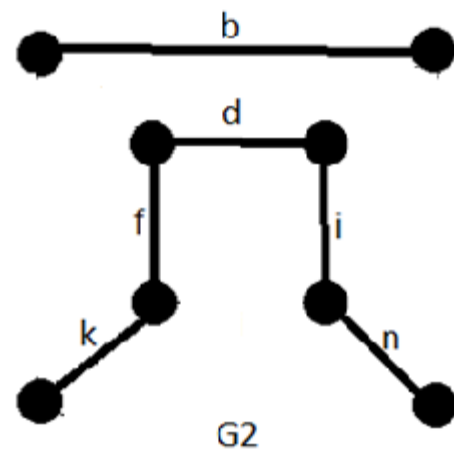
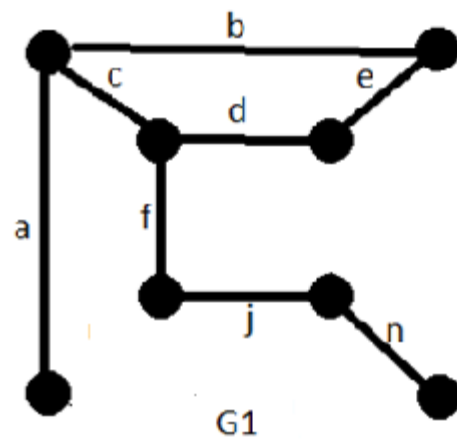
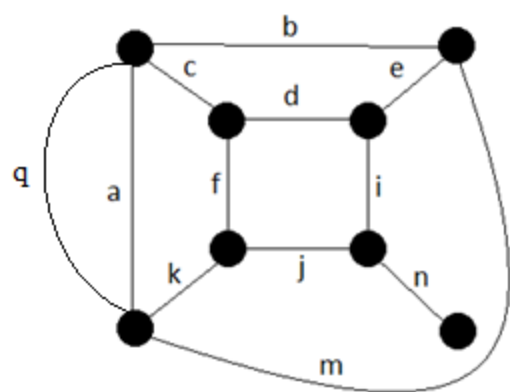
3.1.4 Định lý: Cho T là đồ thị có n đỉnh. Ta có :
 T là cây $\Leftrightarrow T$ liên thông và có $n-1$ cạnh.



3.3 Cây khung (Spanning Tree):

3.3.1 Định nghĩa : *T là cây khung của một đồ thị $G = (V, E)$ nếu T là một **cây** có tập đỉnh là tập đỉnh V và tập cạnh là tập con của E của G.*





3.3.3 Thuật toán tìm cây khung:

Thuật toán 1: **B**readth-**F**irst **S**earch tìm spanning tree

Định nghĩa hàng đợi : Hàng đợi là một tập hợp có tính chất :

- Mỗi lần cho vào hàng đợi một phần tử,
- Mỗi lần chỉ lấy một phần tử ra khỏi hàng đợi,
- Phần tử vào trước sẽ lấy ra trước,

VD : Các phần tử lần lượt đưa vào hàng đợi Q là 1, 4, 2, 3, thì thứ tự lấy ra là 1, 4, 2, 3. **đỉnh hàng đợi**.

3	2	4	1
---	---	---	---

Tìm theo chiều rộng trước (**B**readth-**F**irst **S**earch, **BFS**) :

Tree_BFS(r)

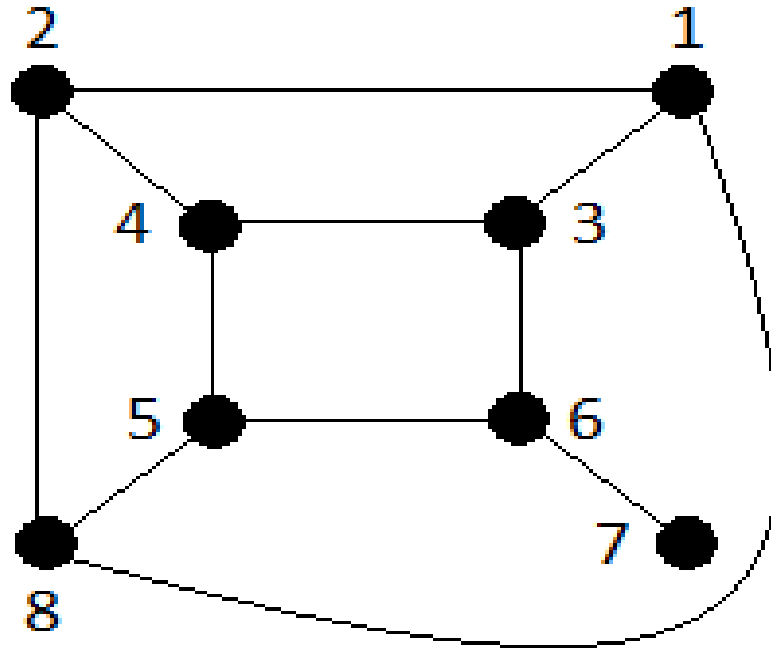
```
{
  QUEUE =  $\emptyset$ ; QUEUE  $\leftarrow$  r; ChuaXet[r] = 0;
  while (QUEUE  $\neq \emptyset$ )
  {
    v  $\leftarrow$  QUEUE;
    for (u  $\in$  Ke(v)) /* u theo thứ tự từ nhỏ đến lớn */
      if (ChuaXet[u]==1)
      {
        QUEUE  $\leftarrow$  u; ChuaXet[u] = 0; T = T  $\cup$  (v, u);
      }
  } /* Kết thúc while (QUEUE  $\neq \emptyset$ )
} /* Kết thúc Tree_BFS
```

main()

```
{
  Nhập đồ thị;
  for (v  $\in$  V)
    ChuaXet[v] = 1;
  T =  $\emptyset$ ;
  Tree_BFS(root);
}
```

root : gốc cây
khung

Ví dụ : root = 1

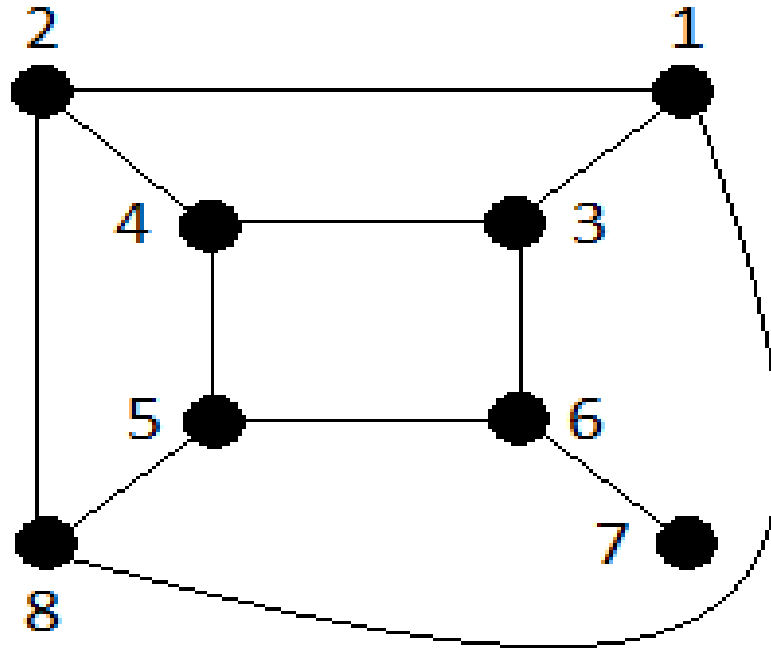


Tìm theo chiều sâu trước (**Depth-First Search** ,**DFS**)

```
void Tree_DFS(v);
{
    ChuaXet[v] = 0;
    for (u ∈ Ke(v))
        /* u theo thứ tự từ nhỏ đến lớn */
        if (ChuaXet[u])
        {
            T = T ∪ (v,u);
            Tree_DFS(u);
        };
} /* Kết thúc Tree_DFS */
```

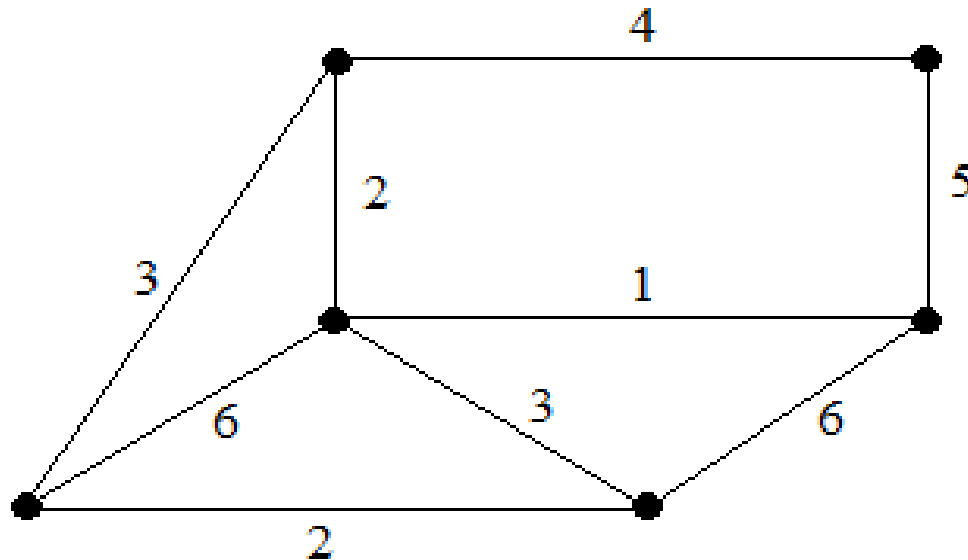
```
main()
{
    for (v ∈ V)
        ChuaXet[v] = 1;
    T = ∅;
    Tree_DFS(root);
}
root : gốc cây khung
```

Ví dụ : root = 1



3.4 Cây khung bé nhất (Minium Spanning Tree):

3.4.1 Định nghĩa: Cho G là đồ thị có trọng số. Cây khung bé nhất là cây khung có tổng các trọng số của các cạnh là bé nhất.



Thuật toán Kruskal :

Kruskal()

{

1. $T = \emptyset$;

2. for $(v \in V)$ **MakeSet**(v); *//Tạo các tập hợp { v }*

3. xếp thứ tự các cạnh trong E tăng dần theo trọng số w;

4. for $(u,v) \in E$ (theo thứ tự đã sắp xếp) :

 if (**FindSet**(u) $\neq$ **FindSet**(v))

// Kiểm tra hai đỉnh u và v có khác nhau ?

 {

$T = T \cup \{(u,v)\}$; *//Chọn cạnh (u,v)*

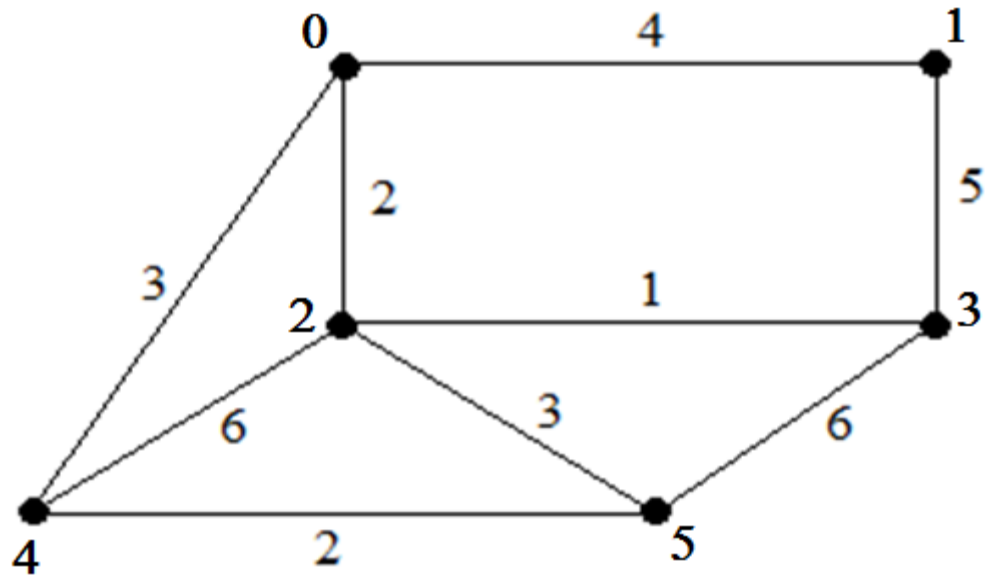
Union(FindSet(u), FindSet(v));

// Gộp tập chứa v và tập chứa u thành một tập

 }

}

Ví dụ: Tìm cây khung bé nhất của đồ thị



Ví dụ: Tìm cây khung bé nhất của đồ thị

Bước	Cạnh	T	Các tập
1, 2, 3	Sắp xếp ↑	$\emptyset$	$\{0\}, \{1\}, \{2\}, \{3\}, \{4\}, \{5\}$
4	(2, 3)		
	(0, 2)		
	(4, 5)		
	(0, 4)		
	(2, 5)		
	(0, 1)		
	(1, 3)		
	(2, 4)		
	(3, 5)		

Thuật toán Prim dạng 1:

- + Cho đồ thị $G=(V,E)$, V là tập các đỉnh, E là tập các cạnh
- + V_T là tập các đỉnh đã được chọn.
- + F là tập các cạnh của cây khung cực tiểu.
- + $w(u,v)$ là trọng số cạnh (u,v)

Bước 1: Khởi tạo

$$F = \emptyset;$$

$$V_T = \{u\}; // \text{Gốc cây khung}$$

Bước 2: Xây dựng cây khung

while ($|F| < n-1$)

{

B1. Chọn $e = \{ w(u,v) \text{ bé nhất, với } (u \in V_T) \ \& \ (v \notin V_T) \}$;

B2. $V_T = V_T \cup \{v\}$;

B3. $F = F \cup \{e\}$;

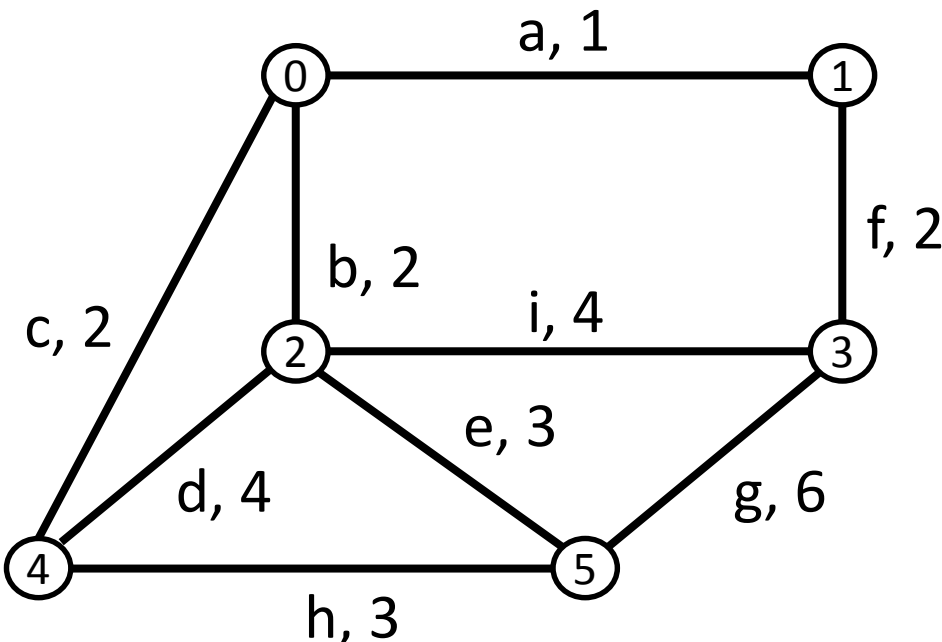
}

Lưu ý :

- u, v ở **B1** được chọn như sau : u là chỉ số bé nhất. Nếu có nhiều v kề u thỏa **B1** thì v bé nhất được chọn.
- Nếu các đỉnh được đánh nhãn là ký tự thì thứ tự theo thứ tự alphabet.

Bài giải: Tìm cây khung bé nhất của đồ thị với gốc = 0.

V_T	F



Thuật toán Prim (Dạng 2).

- Đồ thị không có cạnh song song.
- r : gốc.

Bước 1:

$$S = V$$

Bước 2:

Với mỗi $u \in S$ thực hiện:

{

$$\text{key}[u] = \infty$$

}

Bước 3 :

$$\text{key}[r] = 0$$

$$p[r] = -1$$

Bước 4:

while $S \neq \emptyset$

{ Tìm $u \in S$ với $\text{key}[u]$ bé nhất

$S = S - \{u\}$

Với mỗi $v \in S$ kề với u thực hiện

if ($w_{uv} < \text{key}[v]$)

 { $p[v] = u$

$\text{key}[v] = w_{uv}$

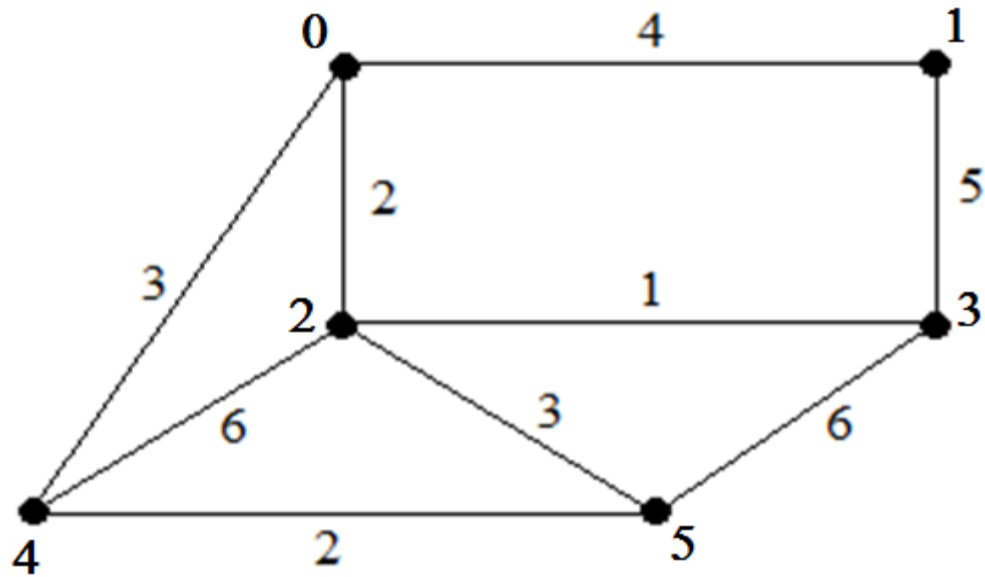
 }

 }

Bước 5:

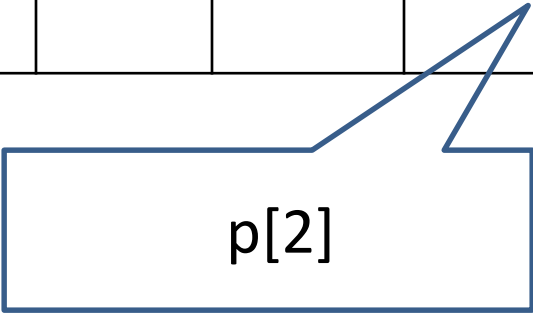
Viết mảng p .

Ví dụ: Tìm cây khung bé nhất của đồ thị



r = 0

Bước	u	S	key[0]	key[1]	key[2]	key[3]	key[4]	key[5]
1		{0, 1, 2, 3, 4, 5}						
2, 3		{0, 1, 2, 3, 4, 5}	0, -1	∞	∞	∞	∞	∞
4	0	{1, 2, 3, 4, 5}		4, 0	2, 0		3, 0	
	2	{1, 3, 4, 5}				1, 2		3, 2
	3	{1, 4, 5}						
	4	{1, 5}						2, 4
	5	{1}						
	1	$\emptyset$						
Kết quả			0, -1	4, 0	2, 0	1, 2	3, 0	2, 4



p[2]

Tài liệu tham khảo:

1. Discrete Mathematics , Richard Johnsonbaugh
2. Algorithms, Thomas h. Cormen
3. Toán Rời Rạc Nâng Cao, Trần Ngọc Danh,
ĐHQG TP HCM
4. Lý Thuyết Đồ Thị, Đặng Trường Sơn, Lê văn
Vinh, ĐHSP Kỹ Thuật TP HCM

Chương 6.

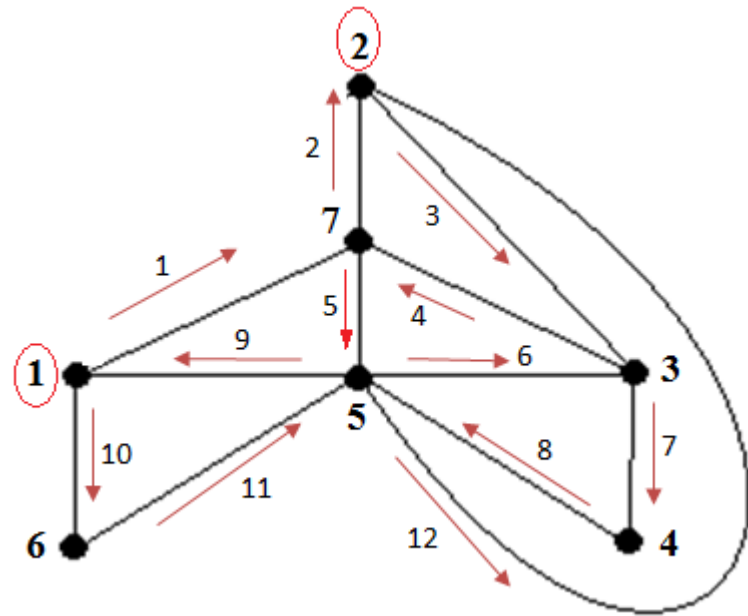
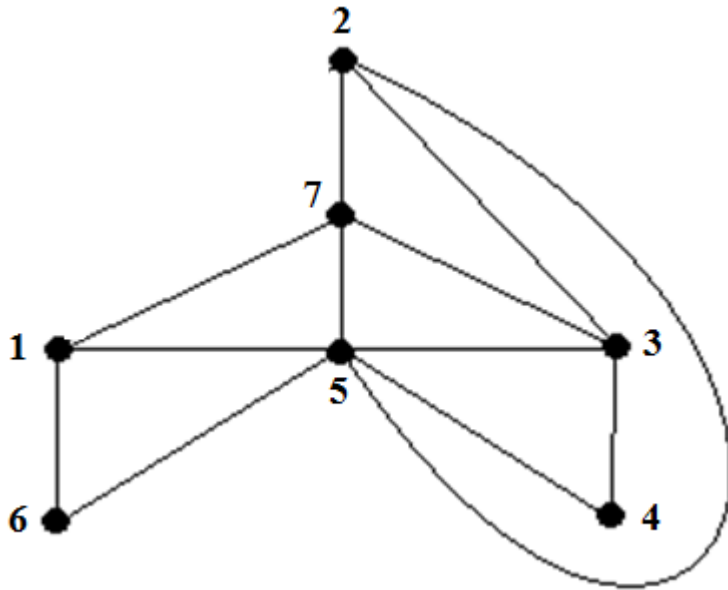
Đường đi Euler, đường đi hamilton

Chu trình Euler , chu trình Hamilton

6.1 Đường đi Euler, Chu trình Euler :

6.1.1 Định nghĩa : Cho $G=(V, E)$. Một đường đi Euler trong G là đường đi chứa toàn bộ cạnh của G và mỗi cạnh xuất hiện 1 lần.

Trong đồ thị có hướng đường đi Euler được định nghĩa tương tự.



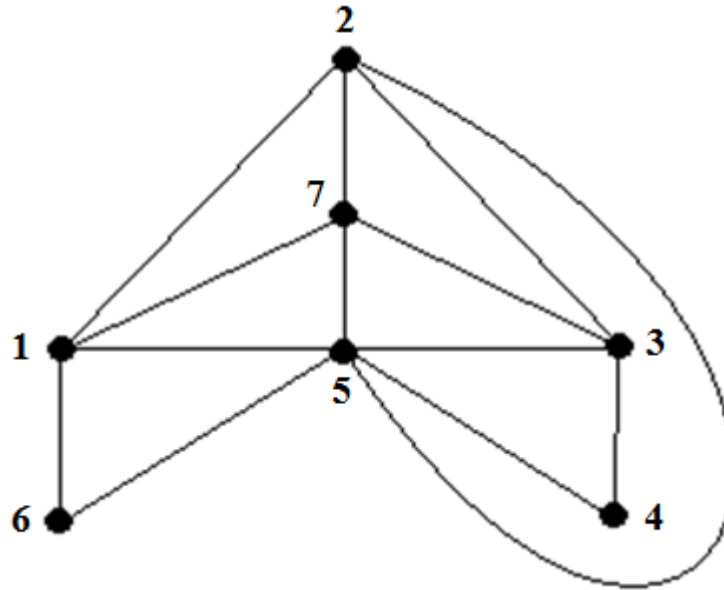
6.1.2 Định nghĩa : Cho $G=(V, E)$. Một chu trình Euler trong G là chu trình chứa toàn bộ cạnh của G và mỗi cạnh xuất hiện 1 lần.

Trong đồ thị có hướng chu trình Euler được định nghĩa tương tự.

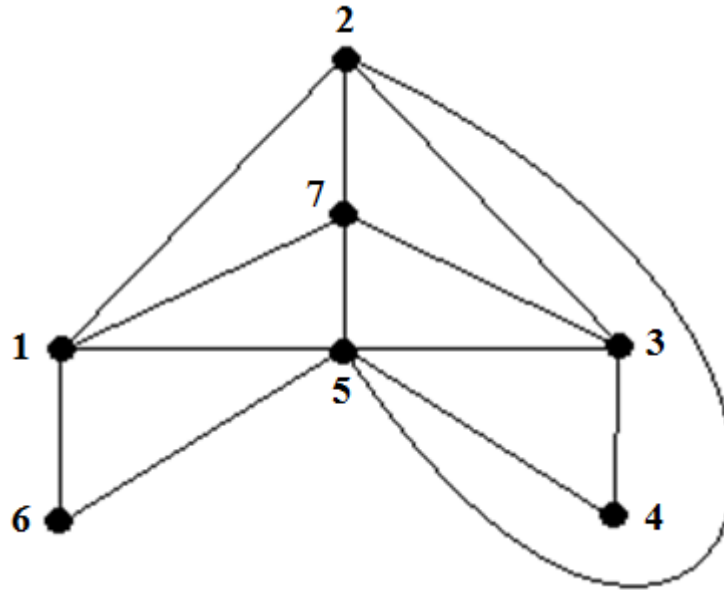
Đồ thị (vô hướng hay có hướng) có chu trình Euler được gọi là đồ thị Euler.

Chú ý : Nếu G chỉ có 1 đỉnh v và không có cạnh ta xem G có một chu trình Euler và ký hiệu (v) .

6.1.2 Định lý : Nếu G (vô hướng) có chu trình Euler thì G liên thông và mọi đỉnh đều có bậc chẵn.

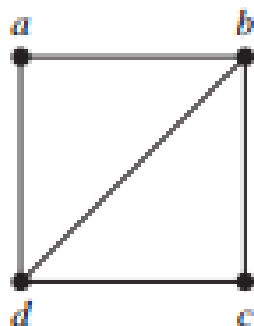
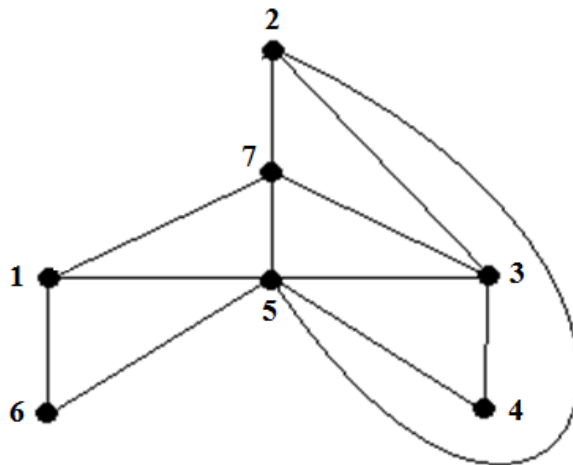


6.1.3 Định lý : Nếu G liên thông và mọi đỉnh của G có bậc chẵn thì G có chu trình E .

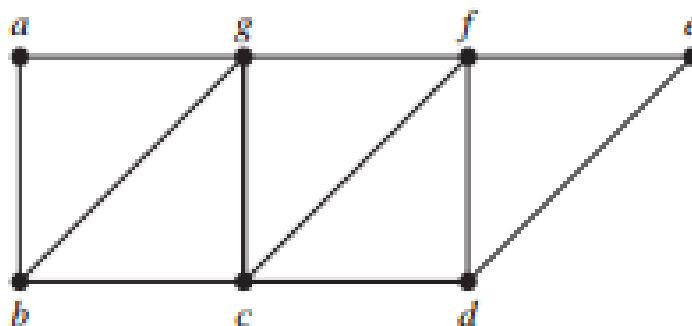


6.1.4 Định lý : Cho G liên thông.

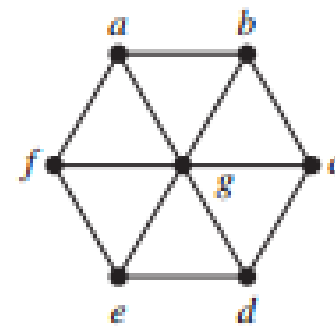
Có đường đi Euler (không phải là chu trình Euler)
khi và chỉ khi G có đúng 2 đỉnh có bậc lẻ.



G_1



G_2



G_3

6.1.5 Thuật toán tìm **chu trình Euler**:

- S : STACK rỗng ban đầu.
- CE : (Stack) tập các cạnh của chu trình Euler, rỗng ban đầu.
- $TopStack(S)$: cho giá trị phần tử của đỉnh STACK.
- $InputStack(S, v)$: cho v vào đỉnh STACK.
- $DeStack(S)$: xóa đỉnh v ở đỉnh STACK.
- $Adj(x)$: Là *danh sách* các đỉnh kề với đỉnh x .
- $Top(Adj(x))$: phần tử đầu của $Adj(x)$.

Bước 1 :

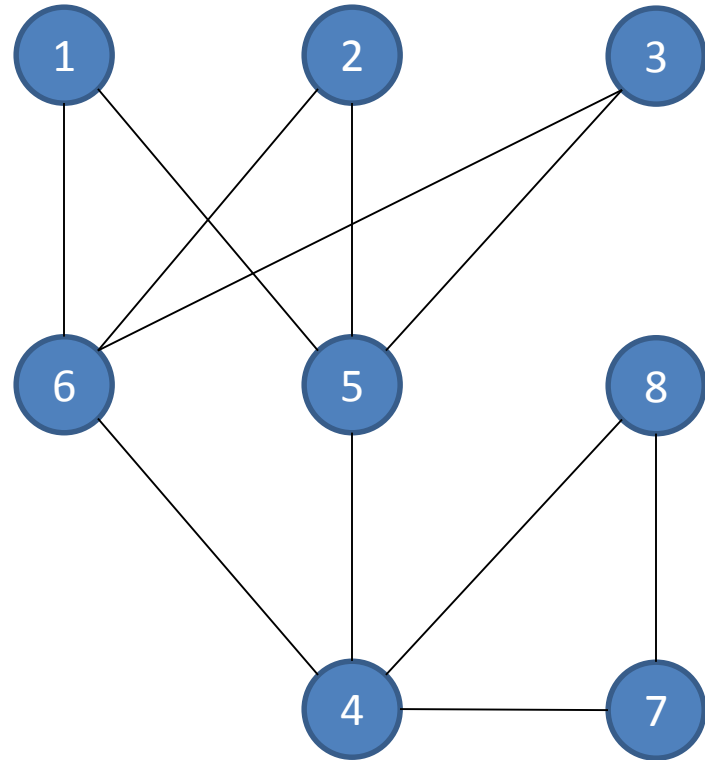
1. **$i = 0$**

2. $u = 3$; // u là đỉnh tùy ý.

3. `InputStack(S, u);`

Bước 2 :

```
while ( $S \neq \emptyset$ )  
{  
  1.  $x = \text{TopStack}(S)$ ;  
  If ( $\text{Adj}(x) \neq \emptyset$ ) {  
    2.  $y = \text{Top}(\text{Adj}(x))$ ;  
    3.  $\text{InputStack}(S, y)$ ;  
    4.  $\text{Adj}(x) = \text{Adj}(x) - \{y\}$ ;  
    5.  $\text{Adj}(y) = \text{Adj}(y) - \{x\}$ ;  
  }  
  Else { 6.  $\text{InputStack}(\text{CE}, x)$ ;  
         7.  $\text{DeStack}(S)$ ;  
  }  
}
```



$u = 3$; InputStack(S, u) ; // u là đỉnh tùy ý.

while ($S \neq \emptyset$)

{ 1. $x = \text{TopStack}(S)$;

If ($\text{Adj}(x) \neq \emptyset$) {

2. $y = \text{Top}(\text{Adj}(x))$;

3. InputStack(S, y);

4. $\text{Adj}(x) = \text{Adj}(x) - \{y\}$;

5. $\text{Adj}(y) = \text{Adj}(y) - \{x\}$;

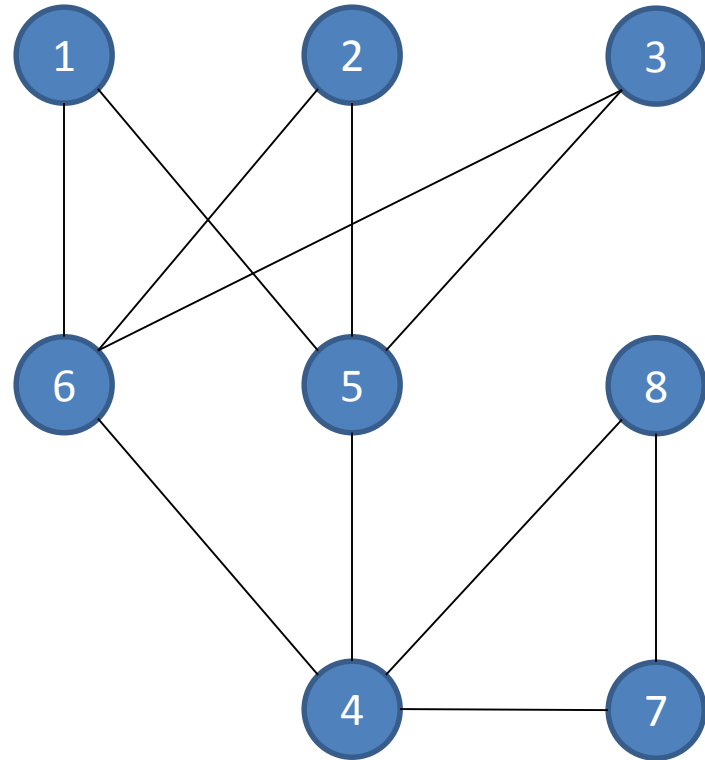
}

Else { 6. InputStack(CE, x);

7. DeStack(S);

}

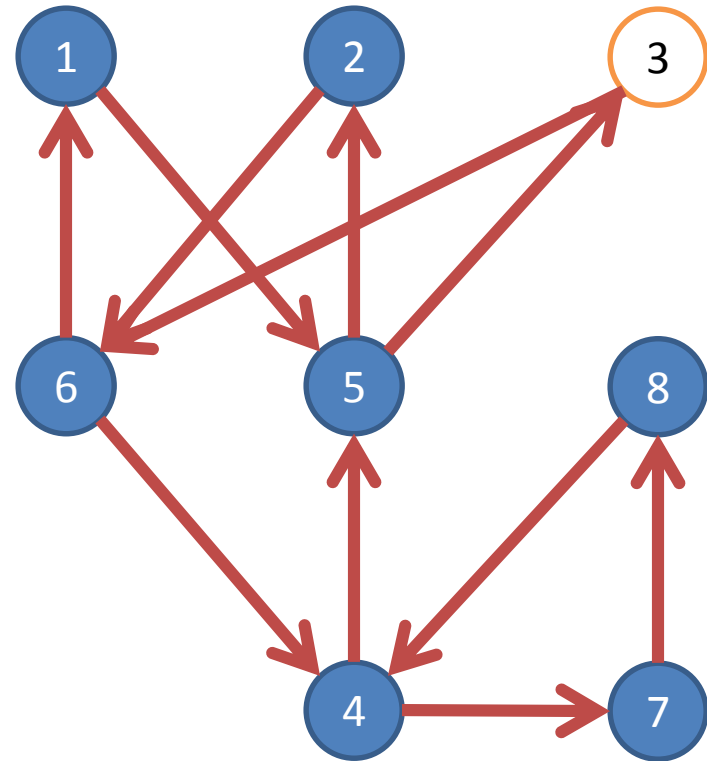
}



$x =$

$y =$

Kết quả :



BÀI TẬP NỘP:

$u = 3$; // u là đỉnh tùy ý.

InputStack(S, u);

while ($S \neq \emptyset$)

{ $x = \text{TopStack}(S)$;

 If ($\text{Adj}(x) \neq \emptyset$) {

$y = \text{Top}(\text{Adj}(x))$;

 InputStack(S, y);

$\text{Adj}(x) = \text{Adj}(x) - \{y\}$;

$\text{Adj}(y) = \text{Adj}(y) - \{x\}$;

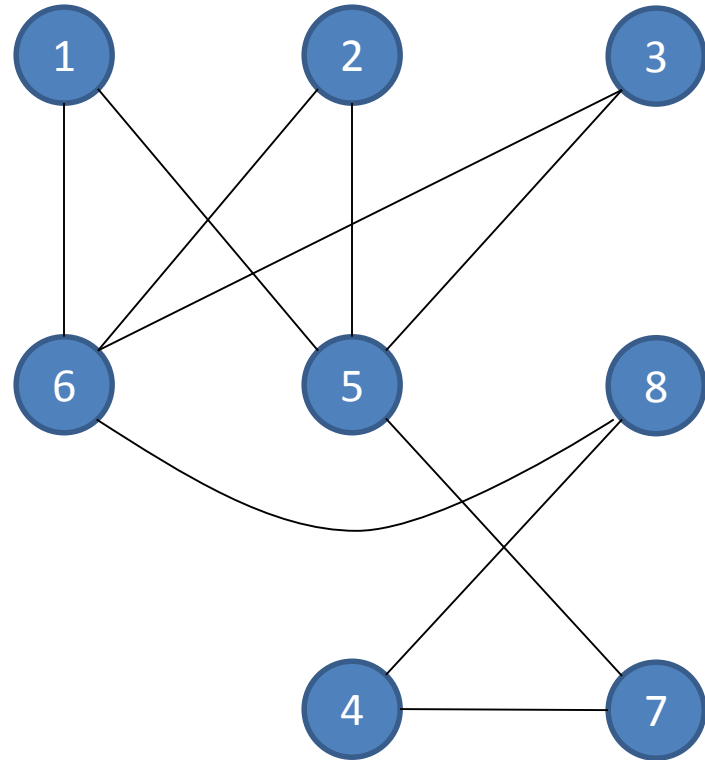
 }

 Else { InputStack(CE, x);

 DeStack(S);

 }

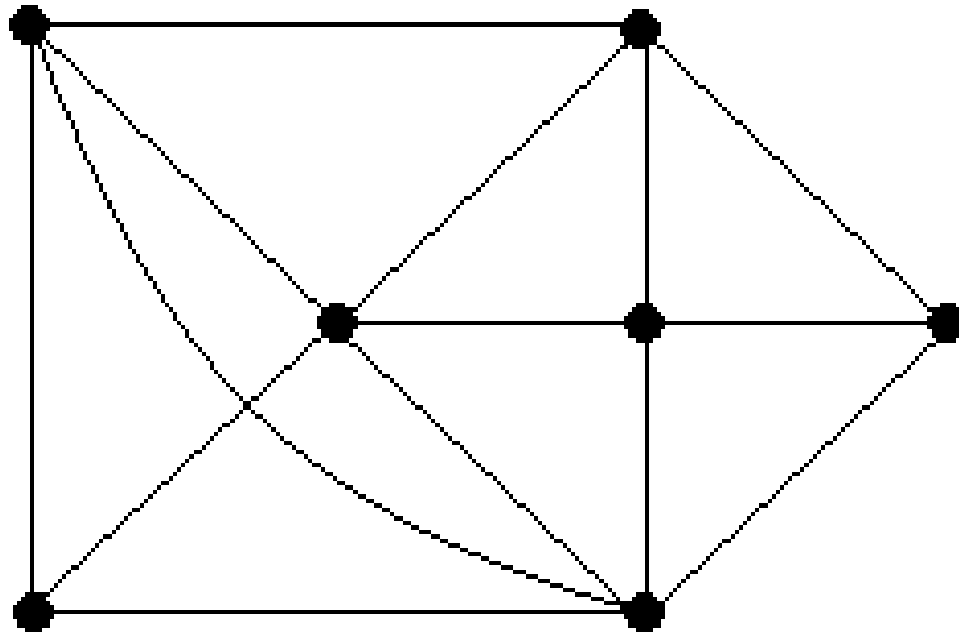
}



6.2 Chu trình Hamilton:

6.2.1 Định nghĩa : Cho $G=(V, E)$ là đồ thị vô (có) hướng . Một chu trình sơ cấp trong G chứa tất cả các đỉnh, được gọi là chu trình Hamilton.

Ví dụ :



6.2.2 Định lý Dirac : Cho $G=(V, E)$ là đồ thị vô hướng có n đỉnh ($n \geq 3$). Nếu mọi $v \in V$, $d(v) \geq n / 2$ thì G có chu trình Hamilton.

6.2.3 Định lý Dirac : Cho $G=(V, E)$ là đồ thị có hướng có n đỉnh . Nếu mọi $v \in V$, $d^+(v) \geq n / 2$ và $d^-(v) \geq n / 2$ thì G có chu trình Hamilton.

6.2.3 Thuật toán tìm chu trình Hamilton:

Thủ tục Hamilton(k)

{

For each $y \in \text{Adj}(x_{k-1})$

{ If $((k=n+1) \ \& \ (y = v_0))$ Print($x_1, x_2, \dots, x_n, v_0$);

Else If $(\text{Color}[y]=\text{White})$ {

$x_k = y;$

$\text{Color}[y] = \text{Gray};$

Hamilton($k+1$);

$\text{Color}[y] = \text{White};$

}

}// For

}

Main()

{

For each $v \in V$: Color[v]=White;

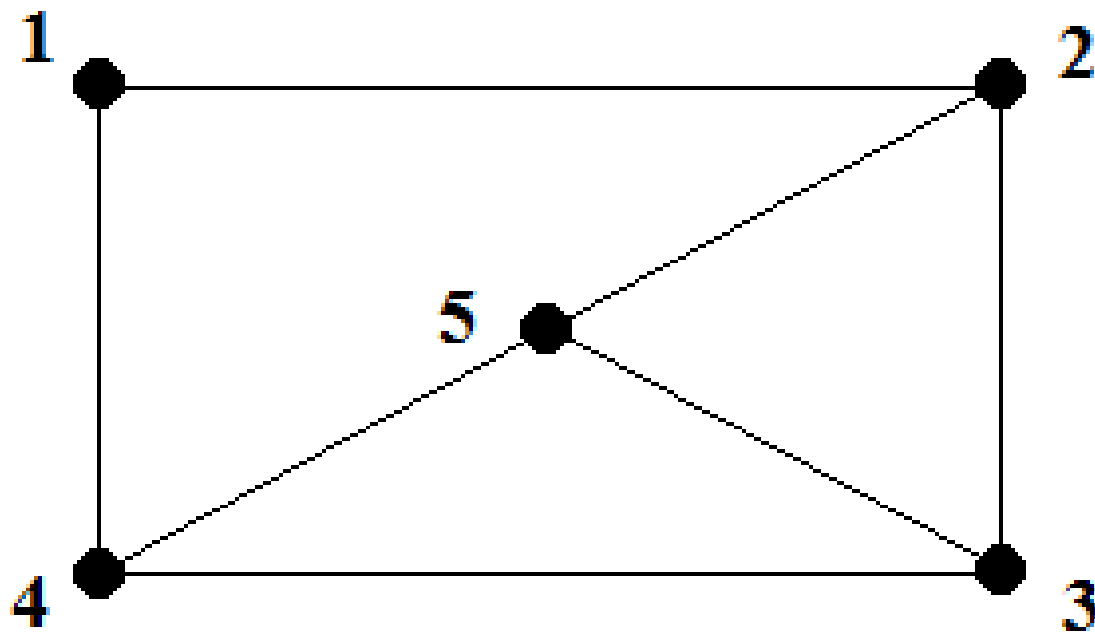
$x_1 = v_0$;

Color[v₀]=Gray;

Hamilton(2);

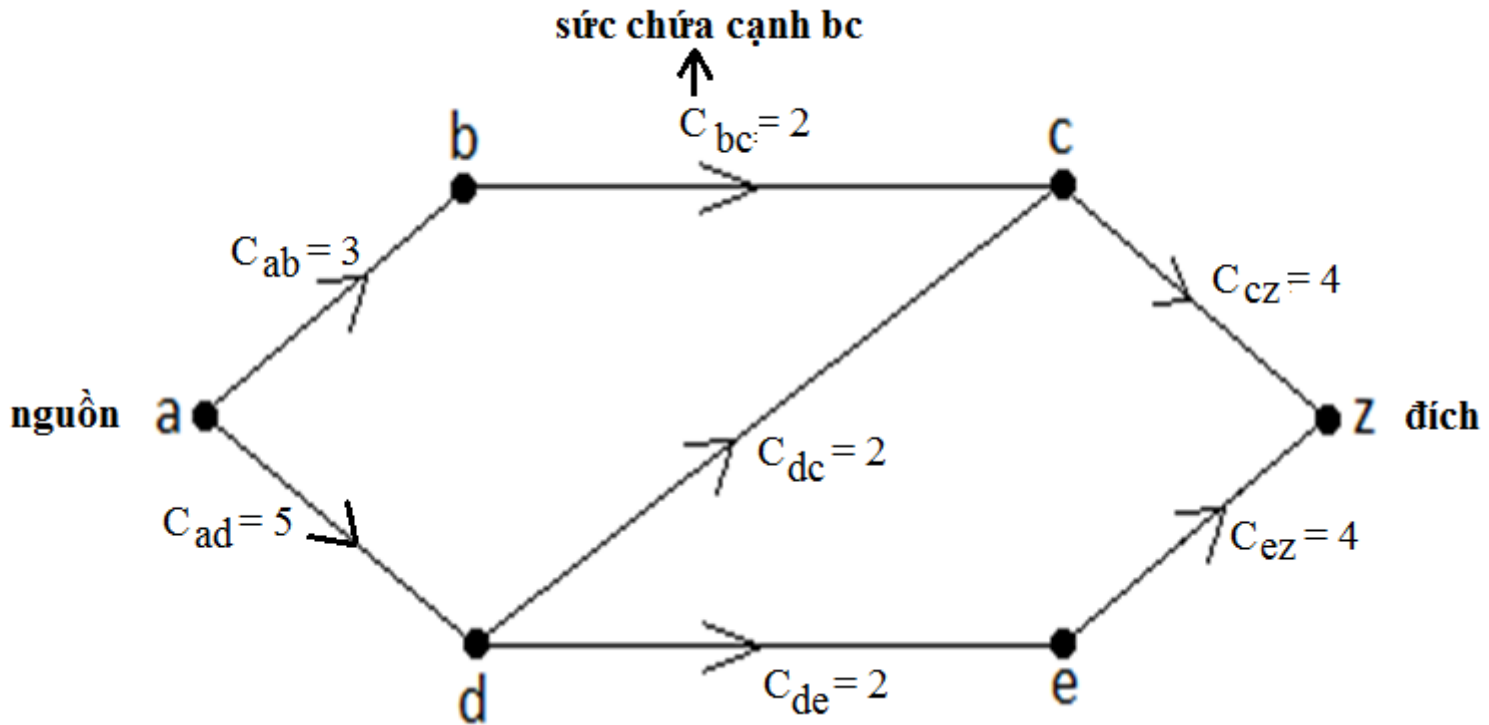
}

Ví dụ :



CHƯƠNG 5 : DÒNG CHẢY LỚN NHẤT

Một mạng vận tải (mạng) là đồ thị G đơn giản, có trọng số.



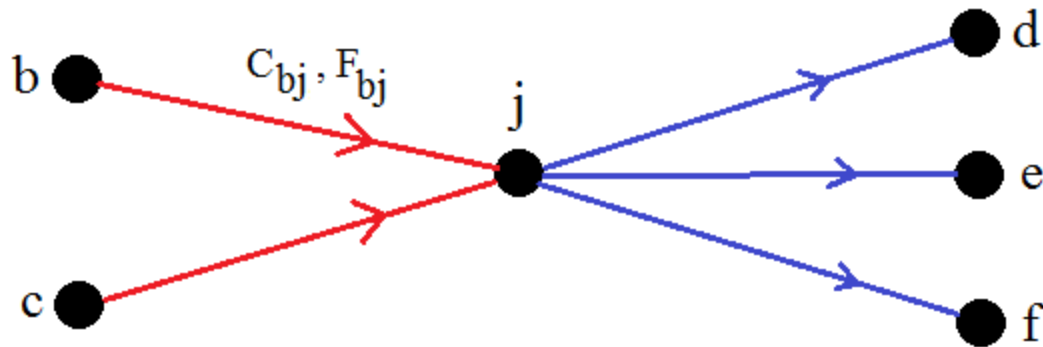
5.1 Dòng chảy lớn nhất:

5.1.1 Định nghĩa:

Một mạng vận tải (mạng) là đồ thị G đơn giản, có trọng số, có hướng thỏa:

- Có đúng 1 đỉnh , được gọi là *nguồn*, không có cạnh vào, đỉnh **a**.
- Có đúng 1 đỉnh , được gọi là *đích* , không có cạnh ra, đỉnh **z**.
- Trọng số C_{ij} của cạnh có hướng (i, j) , được gọi *sức chứa*, là một số không âm.
- Đồ thị G không hướng là liên thông.

Ví dụ :

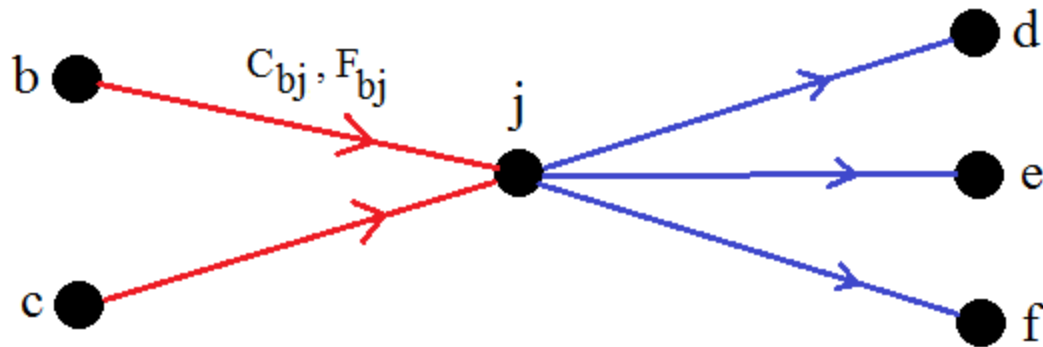


- $F_{bj} \geq 0$

- $F_{bj} \leq C_{bj}$

- $j \notin \{ \mathbf{a}, \mathbf{z} \} \Rightarrow F_{bj} + F_{cj} = F_{jd} + F_{je} + F_{jf}$

Ví dụ :

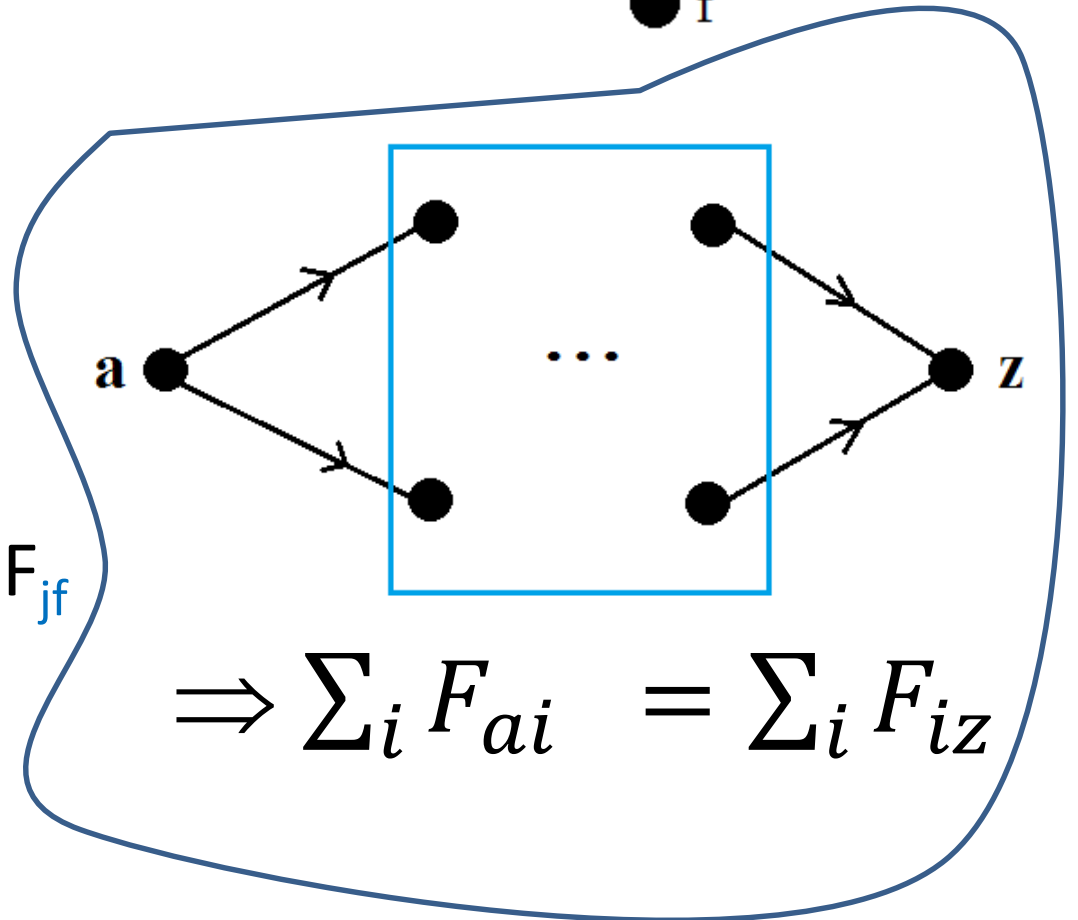


$$- F_{bj} \geq 0$$

$$- F_{bj} \leq C_{bj}$$

$$- j \notin \{a, z\}$$

$$\Rightarrow F_{bj} + F_{cj} = F_{jd} + F_{je} + F_{jf}$$



5.1.2 Định nghĩa:

Cho G là một mạng vận tải. Một *dòng chảy* F trong G gán cho mỗi cạnh có hướng (i, j) một số không âm F_{ij} sao cho:

a) $F_{ij} \leq C_{ij}$.

F_{ij} được gọi là dòng chảy trong cạnh (i, j) .

b) Mỗi đỉnh j , không là nguồn hay đích,

$$\sum_i F_{ij} = \sum_i F_{ji} \quad (1)$$

[Nếu $(i, j) \notin E$ ta cho $F_{ij} = 0$]

Đẳng thức (1) được gọi là **sự bảo toàn của dòng chảy**.

5.1.3 Định lý: Cho F là dòng chảy trong G . Ta có

$$\sum_i F_{ai} = \sum_i F_{iz}$$

Giá trị $\sum_i F_{iz}$ được gọi là *giá trị của dòng chảy F* .

5.1.4 Định nghĩa: Cho G là mạng xét một đường đi không hướng

$$P = (v_0, v_1, \dots, v_n), v_0 = a, v_n = z.$$

Nếu e trong P được định hướng từ v_{i-1} đến v_i (theo G) ta nói e được **định hướng đúng đối với P** , ngược lại e là **không định hướng đúng đối với P** .

5.2 Thuật toán tìm dòng chảy lớn nhất – Thuật toán đánh nhãn:

Input : $a=v_0, v_1, \dots, v_n = z$.

1. $F_{ij} = 0$, với $(i, j) \in E$.
2. Gán nhãn $(, \infty)$ cho a .
3. Nếu z đã được gán nhãn, GOTO bước 6.
4. Chọn đỉnh v_i , ***đã gán nhãn, chưa duyệt***, với chỉ số i bé nhất. **Nếu không có v_i như vậy thì STOP (F là dòng chảy lớn nhất);** ngược lại, $v = v_i$ (v_i **đã duyệt**).

5. Gọi (α, Δ) là nhãn của v . Duyệt mỗi cạnh có dạng $(v, w), (w, v)$ [theo thứ tự $(v, v_0), (v_0, v), (v, v_1), (v_1, v), \dots$], với w ***chưa có nhãn***.

- Với cạnh dạng (v, w) , Nếu $F_{vw} < C_{vw}$, gán nhãn cho w là $(v, \min\{ \Delta, C_{vw} - F_{vw} \})$. Nếu $F_{vw} = C_{vw}$, không gán nhãn cho w .

- Với cạnh dạng (w, v) , nếu $F_{wv} > 0$, gán nhãn cho w là $(v, \min\{ \Delta, F_{wv} \})$. Nếu $F_{wv} = 0$, không đánh nhãn w .

GOTO bước 3.

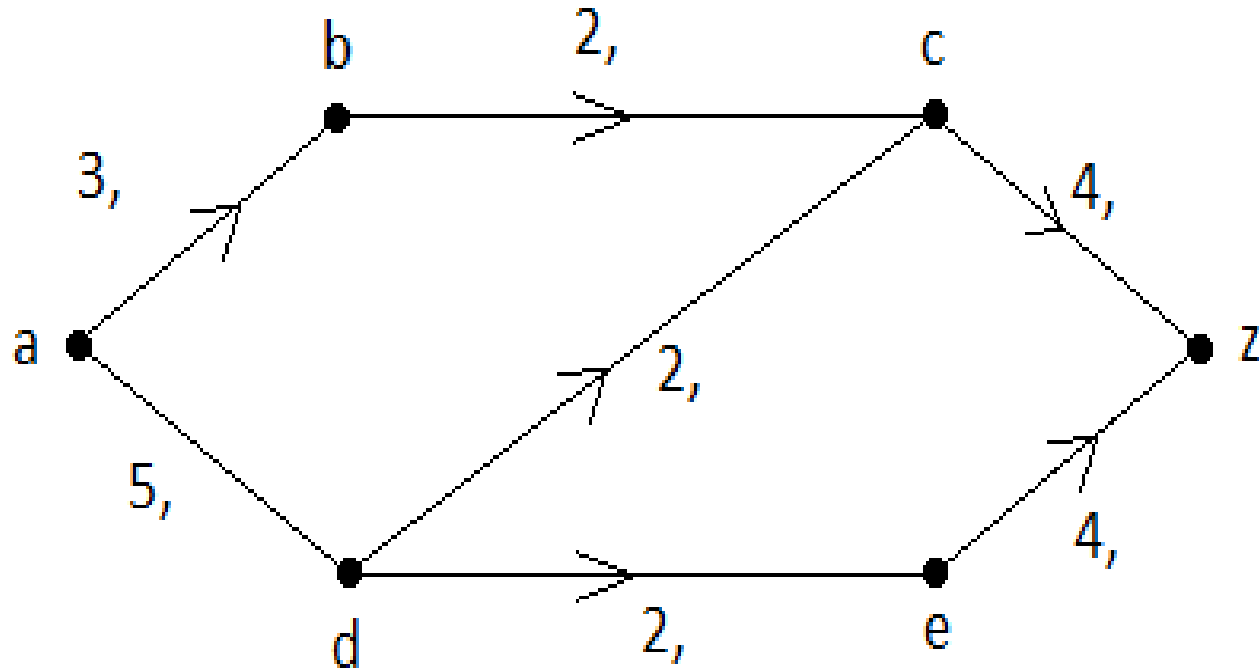
6. Gọi (γ, Δ) là nhãn của z . Gán $w_0 = z$, $w_1 = \gamma$. Nếu nhãn của w_i là (γ', Δ') , gán $w_{i+1} = \gamma'$. Tiếp tục cho đến khi $w_k = a$. Ta có

$$P : a = w_k, w_{k-1}, \dots, w_1, w_0 = z$$

là một đường đi từ a đến z .

- Thay đổi dòng chảy ở mỗi cạnh trong P như sau:
Nếu cạnh e trong P là định hướng đúng tăng dòng chảy của e lên Δ ; ngược lại giảm dòng chảy của e Δ .
- Xóa các nhãn của các đỉnh .
- GOTO bước 2.

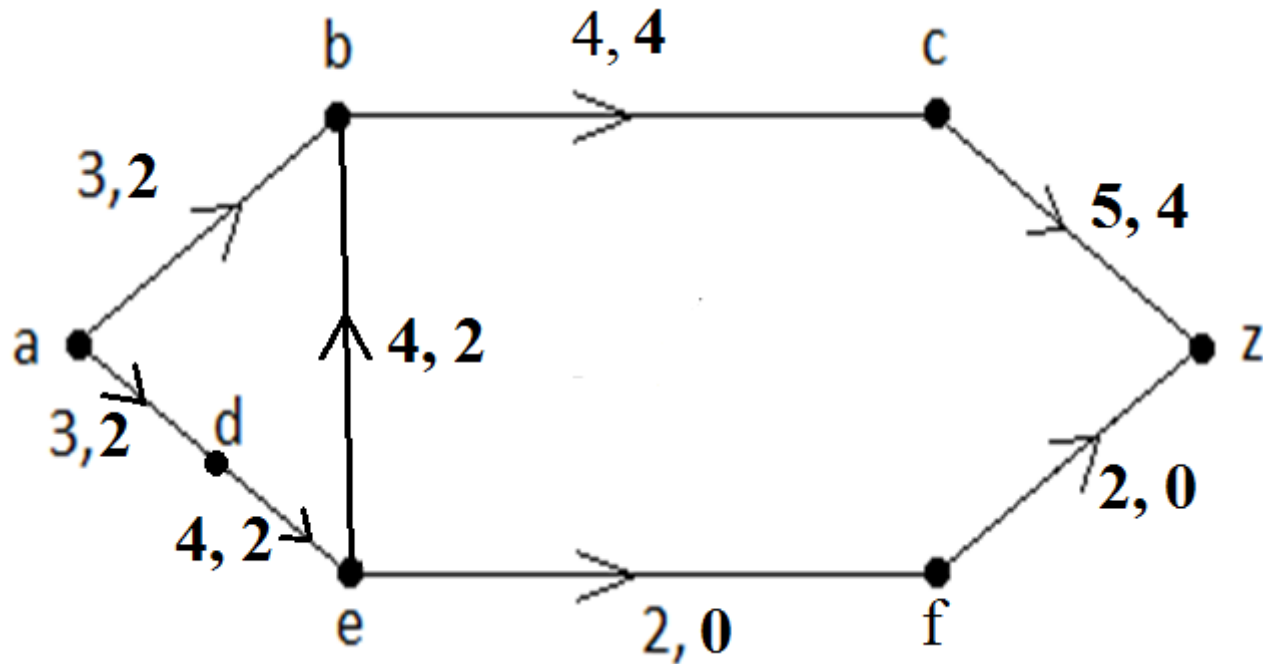
Ví dụ 1 : Các đỉnh sắp theo thứ tự a, b, c, d, e, z.



Bước	v	a	b	c	d	e	z
2		$(, \infty)$					
4	a		$(a, 3)$		$(a, 5)$		
4	b			$(b, 2)$			
4	c						$(c, 2)$
2		$(, \infty)$					

Fab	Fad	Fbc	Fdc	Fde	Fcz	Fez
2	0	2	0	0	2	0

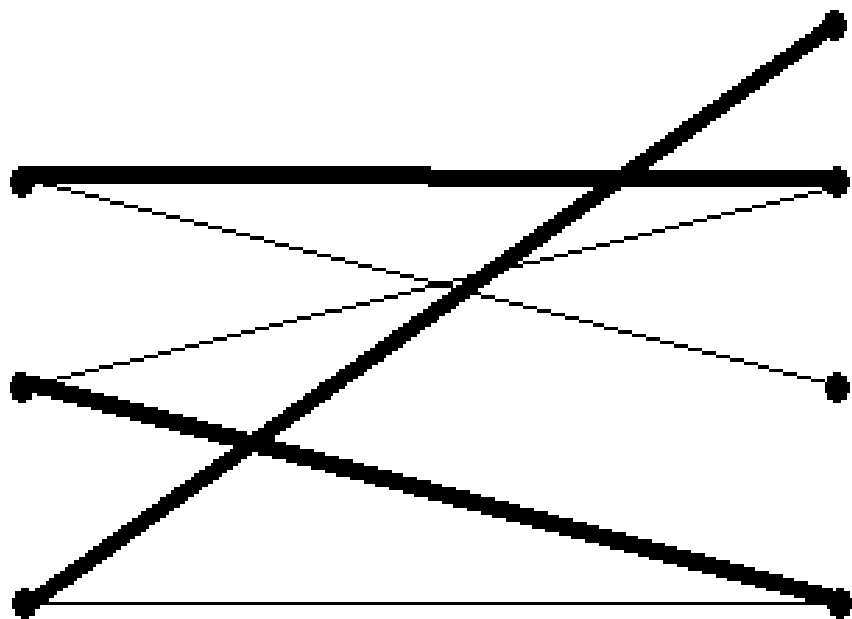
Ví dụ 2 : Các đỉnh sắp theo thứ tự a, b, c, d, e, f, z.



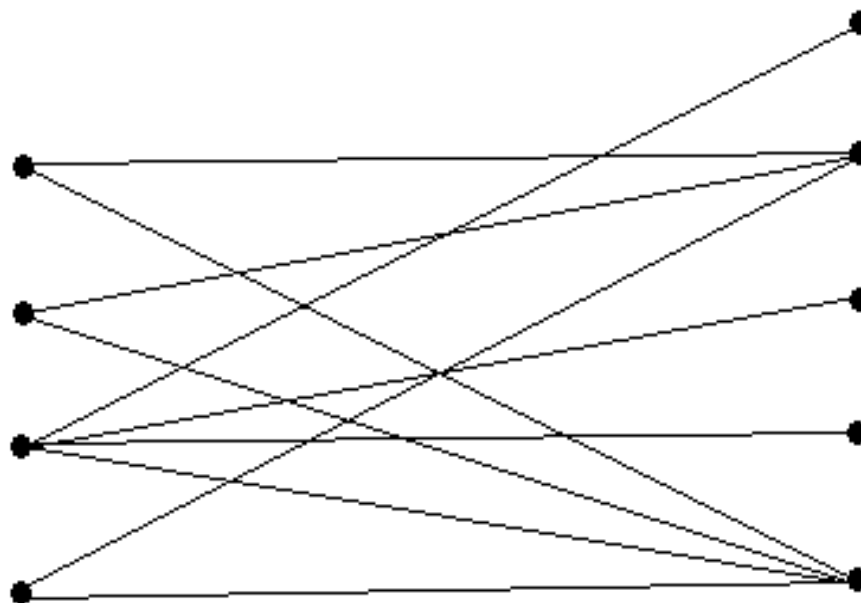
5.3 Bài toán ghép đôi :

5.3.1 Định nghĩa : Cho $G = (V, E)$ là đồ thị có hướng, lưỡng phân với hai tập đỉnh V và W rời nhau. Mỗi cạnh hướng từ V đến W .

- Một ghép đôi trong G là một tập $E' \subseteq E$ các cạnh không có đỉnh chung.
- Một ghép đôi lớn nhất trong G là một ghép đôi E' với số cạnh lớn nhất.
- Một ghép đôi đầy đủ trong G là ghép đôi E với tính chất nếu $v \in V$ thì có $w \in W$, cạnh $(v, w) \in E'$.



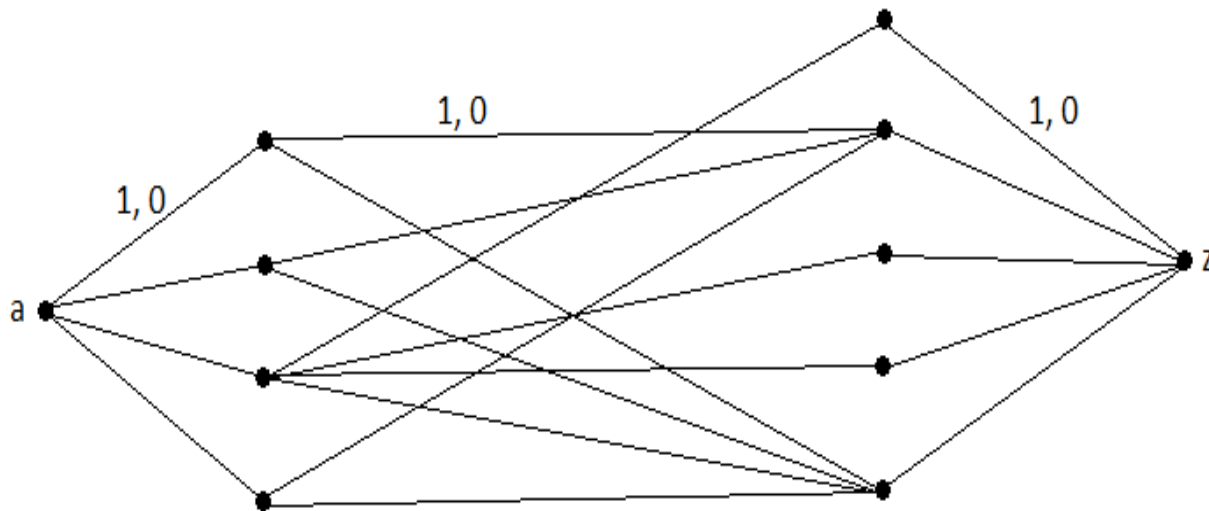
Ví dụ : Tìm một ghép đôi của đồ thị sau :



Giải :

- Gán mỗi cạnh sức chứa bằng 1.
- Thêm nguồn, đích.

Đồ thị vừa được xây dựng được gọi là **matching network**.



5.3.2 Định lý : Cho G có hướng, lưỡng phân với hai tập đỉnh V và W rời nhau, cạnh hướng từ V tới W .

- a) Một dòng chảy trong matching network cho một ghép đôi trong G . $v \in V$ ghép với $w \in W$ khi và chỉ khi dòng chảy trong cạnh (F_{vw}) bằng 1.
- b) Một dòng chảy lớn nhất tương ứng với một ghép đôi lớn nhất.
- c) Một dòng chảy có giá trị bằng $|V|$ tương ứng với một ghép đôi đầy đủ.

Tài liệu tham khảo:

1. Discrete Mathematics , Richard Johnsonbaugh
2. Algorithms, Thomas h. Cormen
3. Toán Rời Rạc Nâng Cao, Trần Ngọc Danh, ĐHQG TP HCM
4. Lý Thuyết Đồ Thị, Đặng Trường Sơn, Lê văn Vinh, ĐHSP Kỹ Thuật TP HCM

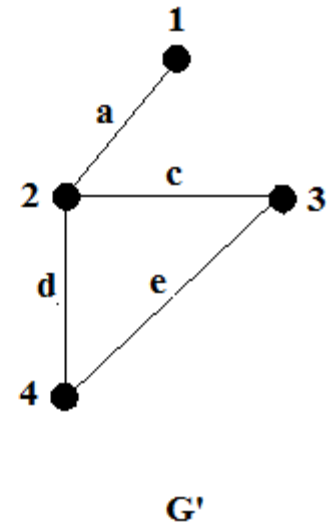
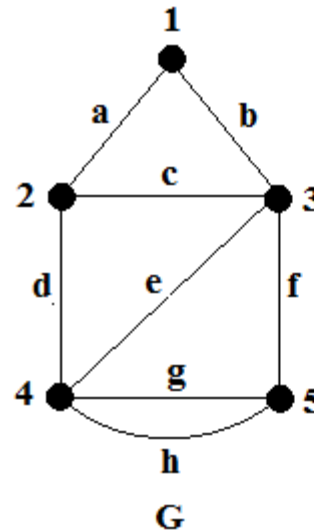
Chương 4. Đồ thị con , đẳng cấu

4.1 Đồ thị con :

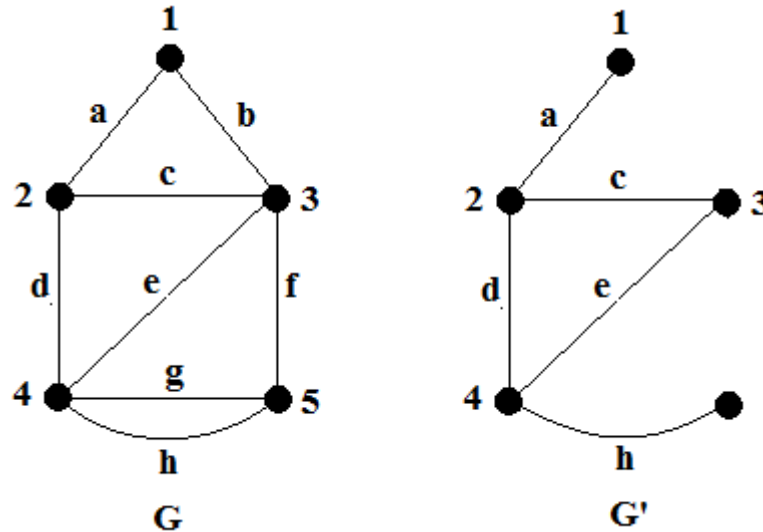
4.1.2 Định nghĩa : Cho $G=(V, E)$ và $G'=(V', E')$ là hai đồ thị cùng có hướng hoặc cùng không hướng. G' được nói là đồ thị con của G , ký hiệu $G' \leq G$, nếu

a) $V' \subseteq V$ và $E' \subseteq E$.

b) $\forall e' = (v', w') \in E'$
 $\Rightarrow v', w' \in V'$.

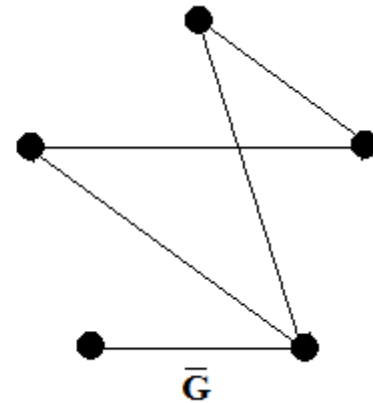
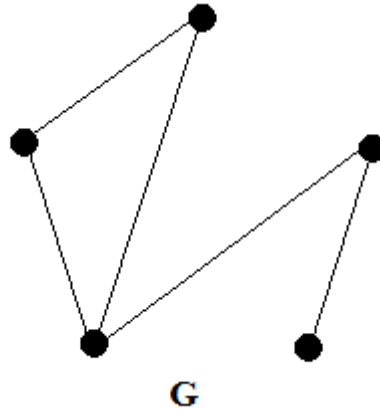
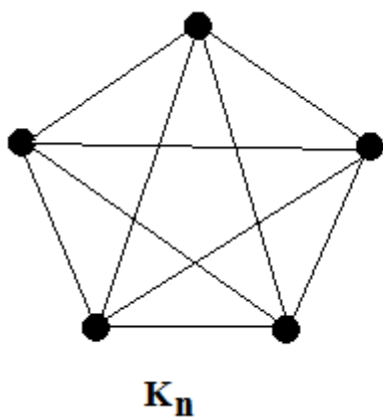


4.1.3 Định nghĩa : Cho $G'=(V', E') \leq G=(V, E)$. G' được nói là *đồ thị khung* của G , nếu $V'=V$.



4.1.4 Định nghĩa : Cho $K_n=(V, E_n)$ là đồ thị , được gọi là đồ thị đầy đủ, có n đỉnh, không có cạnh song song, không có vòng, mỗi đỉnh kề với $n-1$ đỉnh còn lại của đồ thị.

4.1.5 Định nghĩa : Cho $K_n=(V, E_n)$ và $G=(V, E)$ là đồ thị khung của K_n . Đặt $\bar{G} = (V, \bar{E})$ với $\bar{E} = E_n - E$, thì $\bar{G}$ được gọi là đồ thị bù của G .



4.1.5 Bài toán *Instant Insanity* (ứng dụng đồ thị con):

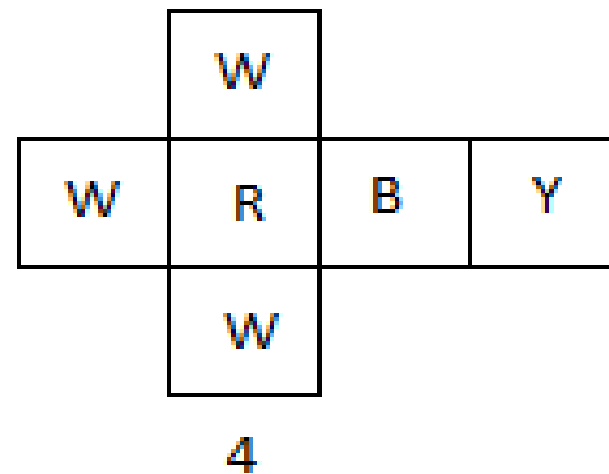
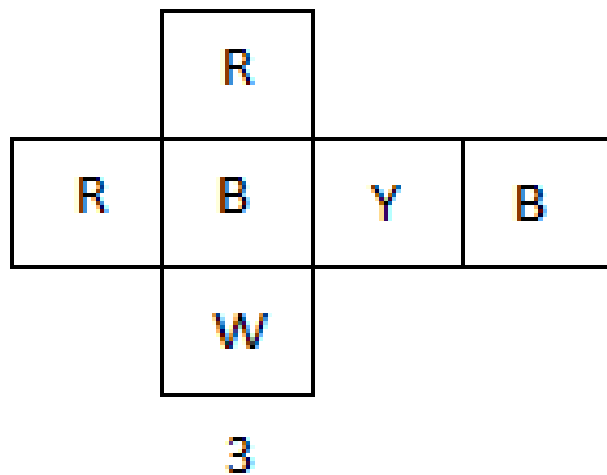
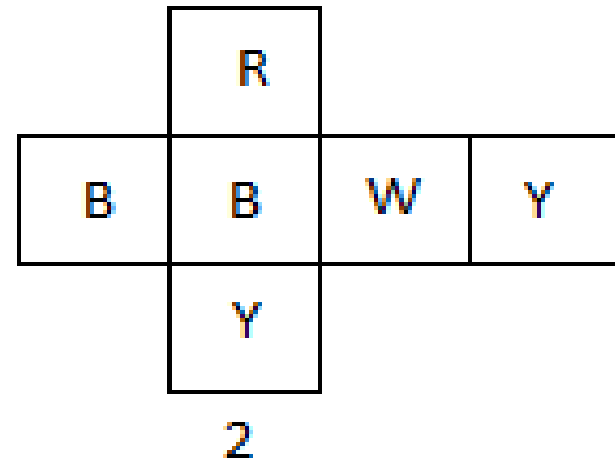
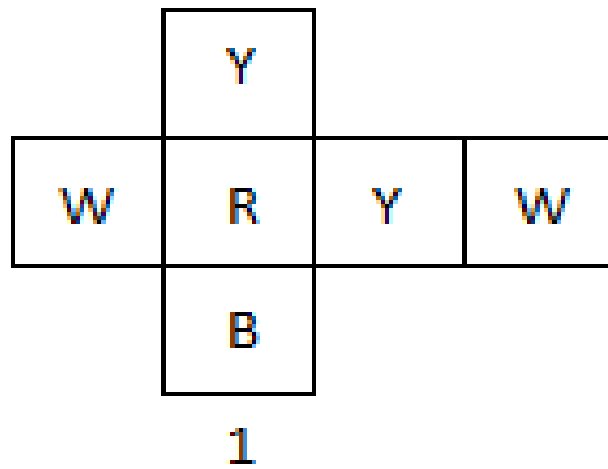
- Cho 4 khối lập phương được đánh số 1, 2, 3, 4.
- Sáu mặt của mỗi khối được tô bằng 4 màu B, R, Y, W.

⇒ Có cách nào chồng 4 khối lên nhau thành 1 cột sao cho không có màu nào xuất hiện 2 lần ở mỗi mặt bên hay không?

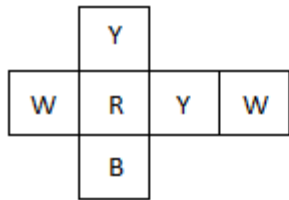
4.1.5 Bài toán *Instant Insanity* (ứng dụng đồ thị con):

R	R	W	Y
W	B	B	B
B	W	Y	W
Y	Y	R	R

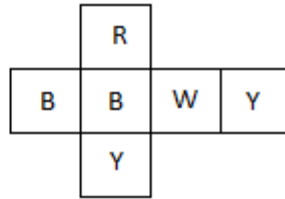
Ví dụ :



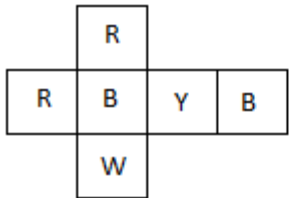
Ví dụ :



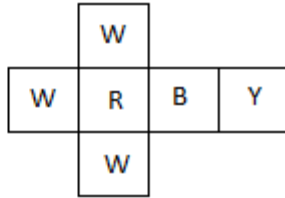
1



2



3

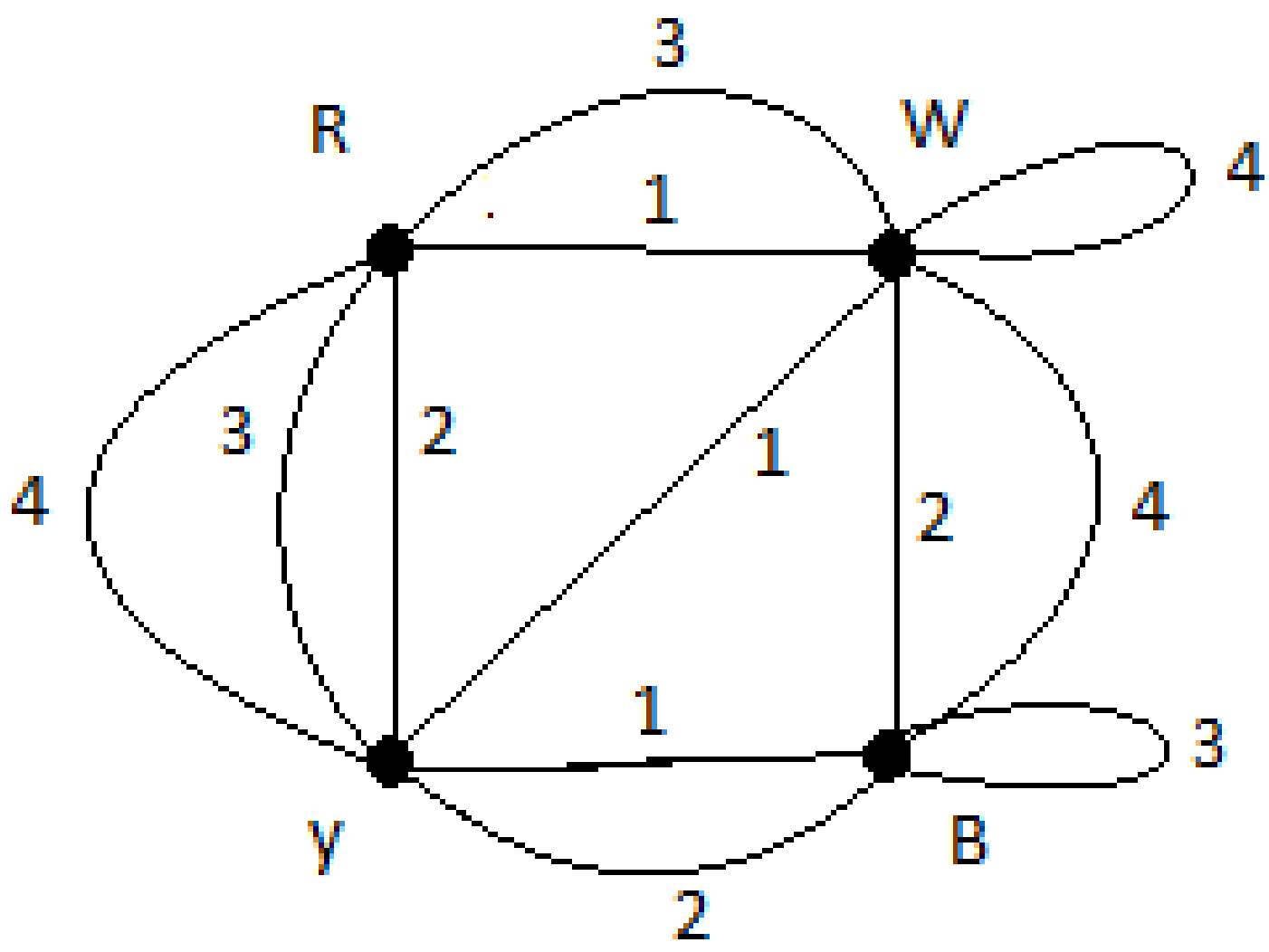


4

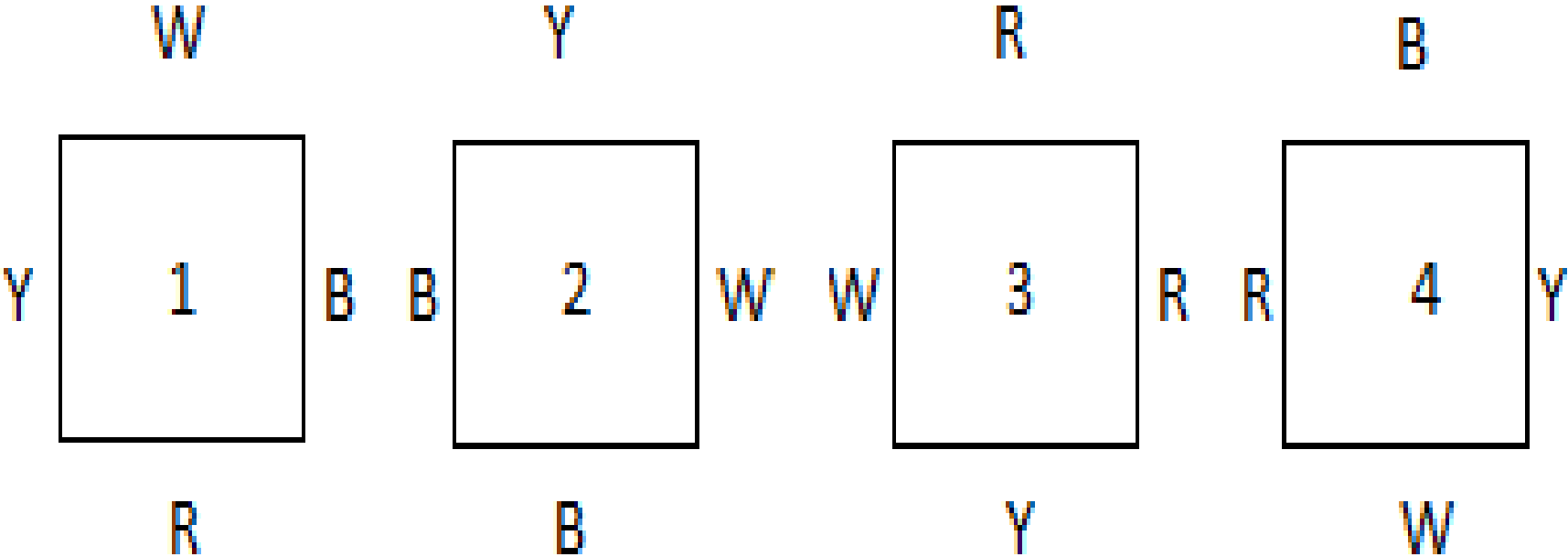
Giải :

Bước 1 : Vẽ đồ thị biểu diễn bốn khối. Với các đỉnh là các màu. Hai đỉnh v và w kề nhau với cạnh có nhãn là i nếu khối i có v và w đối nhau.

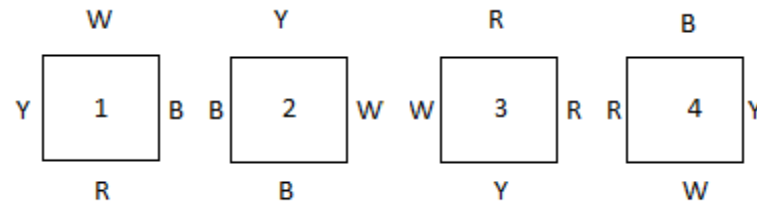
Bước 1 :



Nghiem :

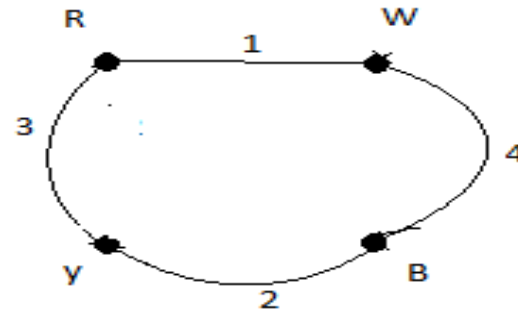
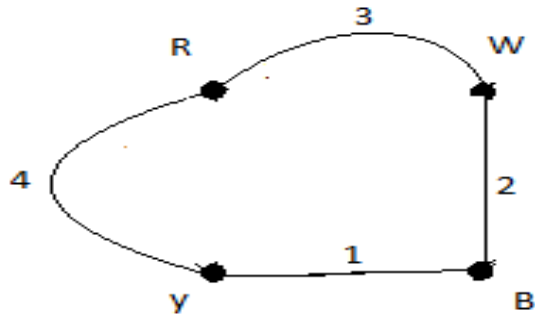


Nghiệm :



Bước 2 : Tìm hai đồ thị con G1 và G2 thỏa :

- Mỗi đỉnh có bậc là 2.
- Hai đồ thị không có cạnh chung (một cạnh dùng 2 lần).
- Mỗi khối thể hiện đúng 1 lần trong mỗi đồ thị con.



⇒ G1 và G2 là nghiệm.

4.2 Đồng cấu:

4.2.1 Định nghĩa : Cho $G_1=(V_1, E_1)$ và $G_2=(V_2, E_2)$. G_1 và G_2 được nói là đồng cấu , ký hiệu $G_1 \sim G_2$, nếu có

- song ánh $f : V_1 \rightarrow V_2$
- song ánh $g : E_1 \rightarrow E_2$

sao cho :

Nếu $e=(v, w) \in E_1$ thì $g(e)=(f(v), f(w)) \in E_2$. **(1)**

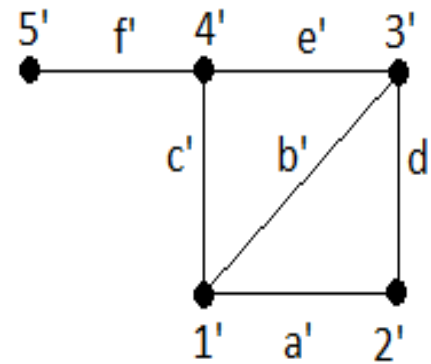
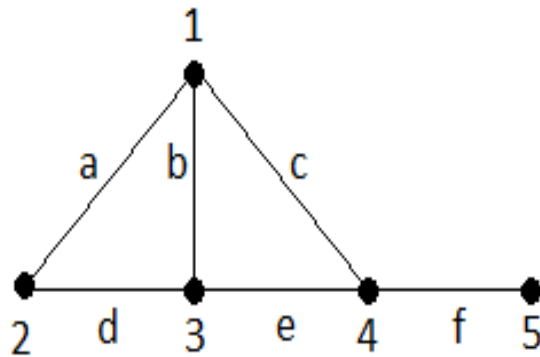
Cặp (f, g) được gọi là một đồng cấu của G_1 lên G_2 .

4.2 Đồng cấu:

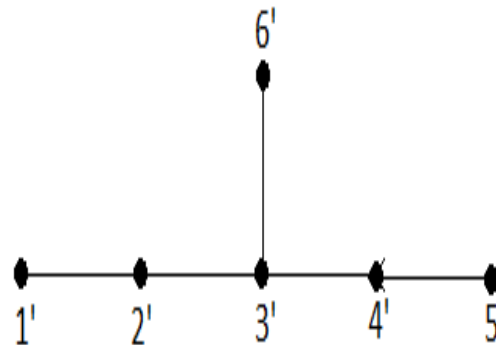
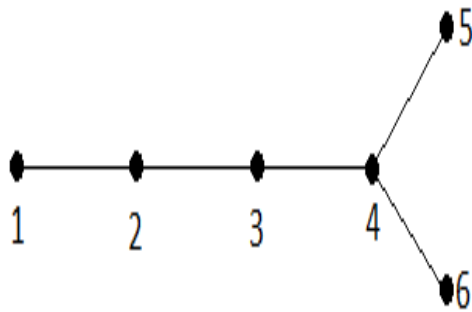
4.2.2 Nhận xét : Cho $G_1=(V_1, E_1)$ và $G_2=(V_2, E_2)$ là đồng cấu thì hai đồ thị

- Có cùng số đỉnh, tức là $|V_1| = |V_2|$,
- Có cùng số cạnh, tức là $|E_1| = |E_2|$,
- Có cùng số đỉnh với bậc cho sẵn,
- Số đỉnh kề với đỉnh $v \in V_1$ và $f(v) \in V_2$ là như nhau.

Ví dụ: Hai đồ thị sau là đẳng cấu.



Ví dụ : Hai đồ thị sau không đẳng cấu:



4.2.3 Định lý : Nếu $G_1 \sim G_2$ thì $G_2 \sim G_1$.

4.2.4 Định lý (Đồng cấu và ma trận kề): Cho G_1 và G_2 là **đơn giản**, $G_1 \sim G_2$ *khi và chỉ khi* có một thứ tự của các đỉnh của **ma trận kề** của G_1 và G_2 sao cho 2 ma trận này là bằng nhau.

➤ **Chú ý :** Khi hoán vị hai cột (dòng) của ma trận kề của G thì ma trận sau hoán vị vẫn là ma trận kề của G .

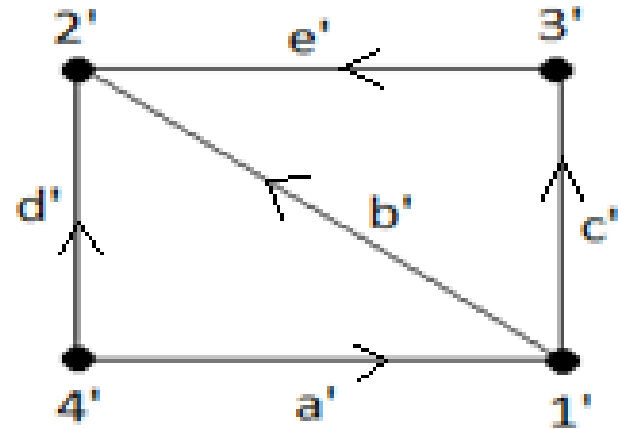
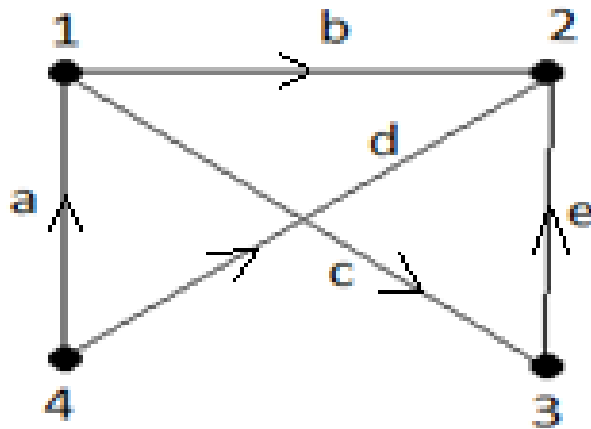
4.2.5 Định nghĩa đẳng cấu cho đồ thị G có hướng:

Cho $G_1=(V_1, E_1)$ và $G_2=(V_2, E_2)$ là hai đồ thị có hướng.

$G_1 \sim G_2$ nếu :

- có song ánh $f : V_1 \rightarrow V_2$,
- có song ánh $g : E_1 \rightarrow E_2$,
- nếu $e=(v, w) \in E_1$ thì $g(e)=(f(v), f(w)) \in E_2$. **(1)**

Ví dụ :



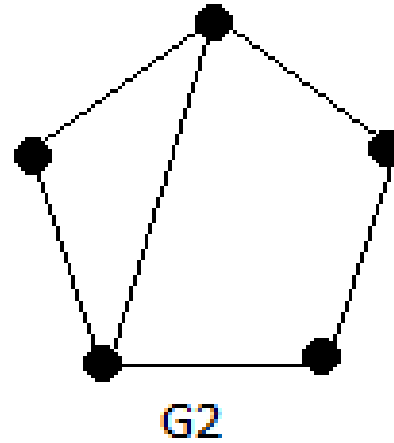
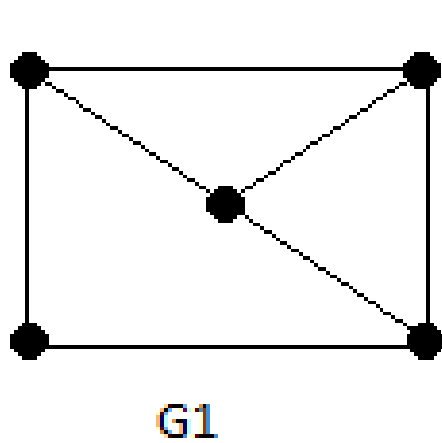
- Sau đây là một cách để kiểm tra 2 đồ thị đơn giản G_1 và G_2 **không đẳng cấu**. Tìm tính chất P mà với mọi đồ thị G_1, G_2

$$P \in G_1, \text{ và } G_1 \sim G_2 \Rightarrow P \in G_2$$

Vậy với G_1 và G_2 đã cho nếu $P \notin G_2$ thì G_1 và G_2 không đẳng cấu.

➤ Tính chất P được gọi là một **bất biến** (invariant) .

Ví dụ 1 : Hai đồ thị G_1 và G_2 sau đây không đẳng cấu, vì G_1 có 7 cạnh trong khi G_2 chỉ có 6 cạnh. Ta nói “**có 7 cạnh**” là một bất biến.

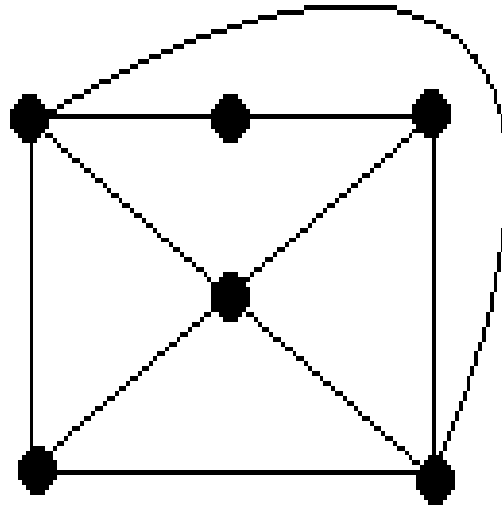


Ví dụ 2 : Cho $k > 0$, “ **có một đỉnh có bậc k** ” là một bất biến.

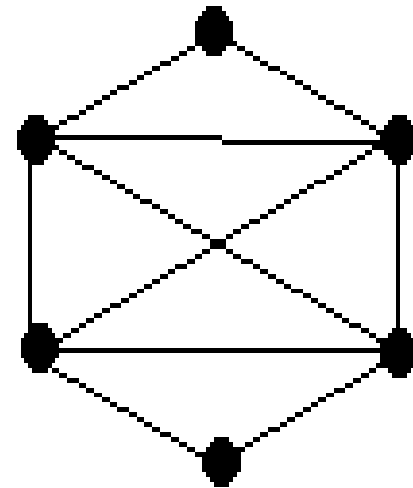
Giả sử G_1 và G_2 là đẳng cấu và f là song ánh $V_1 \rightarrow V_2$. Giả sử G_1 có đỉnh v có bậc k . Khi đó có k cạnh $e_1, \dots, e_k$ kề v . Từ định nghĩa đẳng cấu ta có $g(e_1), \dots, g(e_k)$ kề $f(v)$. Vì g là **đơn ánh** nên $d(f(v)) \geq k$.

Gọi e' là cạnh kề với $f(v)$ trong V_2 . Vì g là **toàn ánh** nên có e trong G_1 sao cho $g(e)=e'$. Vì $g(e)$ kề với $f(v)$ trong G_2 , từ định nghĩa 2.3.1 ta có e kề với v trong G_1 . Vậy $e \in \{e_1, \dots, e_k\}$. Suy ra $d(f(v))=k$.

Xét G_1 và G_2 cho sau đây, vì “ có 1 đỉnh có bậc 3” là bất biến nên G_1 và G_2 không đẳng cấu. G_1 có a và f là bậc 3 trong khi G_2 không có đỉnh nào là bậc 3.



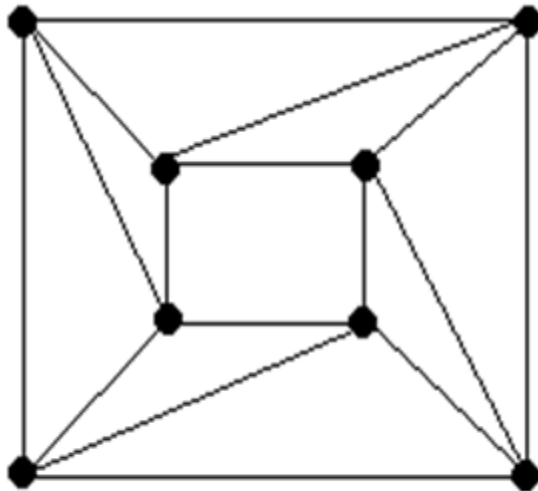
G_1



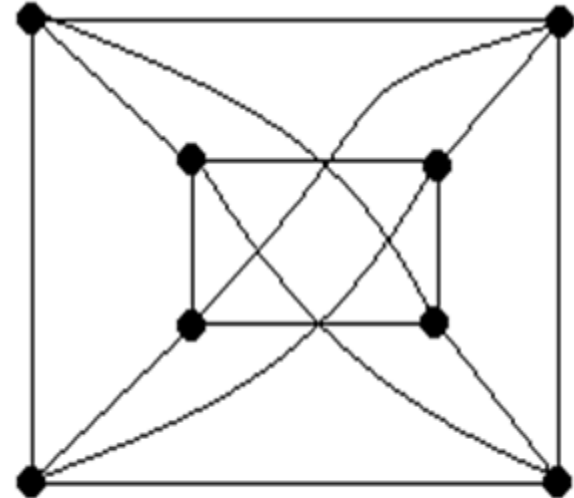
G_2

Ví dụ 3 : “có một chu trình đơn giản có chiều dài k ”
là một bất biến (BT).

Xét G_1 và G_2 cho sau đây là không đẳng cấu. “có một chu trình đơn giản có chiều dài 3” . Vì G_1 có một chu trình chiều dài 3 nhưng G_2 chu trình có chiều dài ít nhất là 4.



G_1



G_2

Tài liệu tham khảo:

1. Discrete Mathematics , Richard Johnsonbaugh
2. Algorithms, Thomas h. Cormen
3. Toán Rời Rạc Nâng Cao, Trần Ngọc Danh,
ĐHQG TP HCM
4. Lý Thuyết Đồ Thị, Đặng Trường Sơn, Lê văn
Vinh, ĐHSP Kỹ Thuật TP HCM

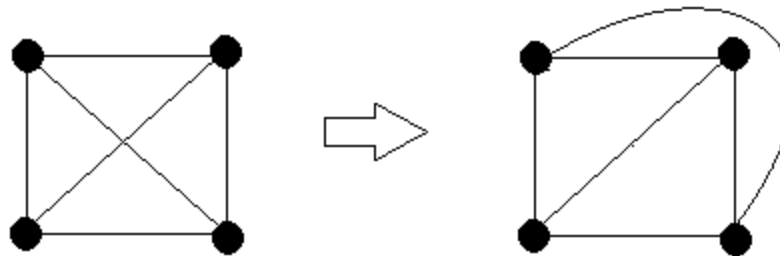
Chương 7. Đồ thị phẳng-Tô màu đồ thị

7.1 Đồ thị phẳng:

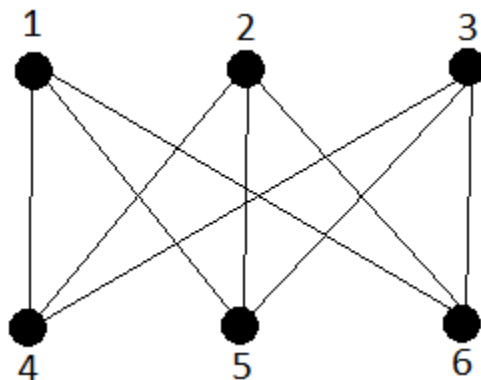
7.1.1 Định nghĩa đồ thị phẳng (*planar graph*):

Một đồ thị G (vô hướng) được nói là phẳng nếu G có thể được vẽ mà các cạnh không cắt nhau.

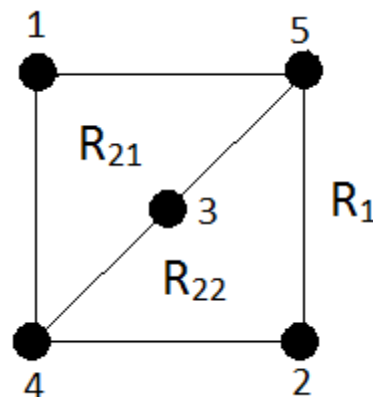
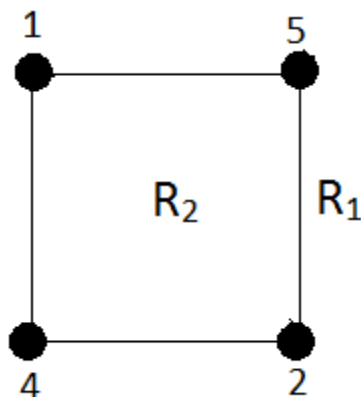
Ví dụ 1 :



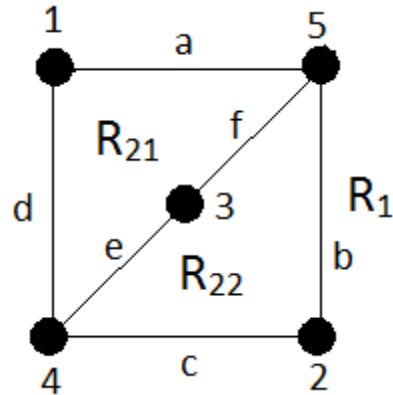
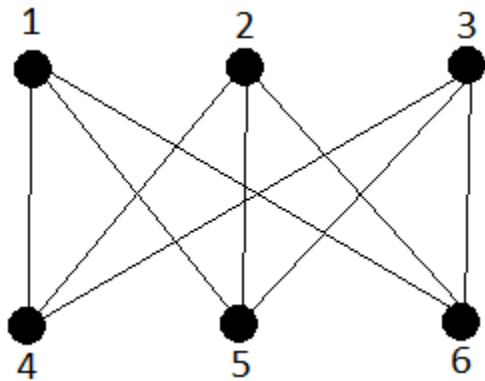
Ví dụ 2 : Đồ thị $K_{3,3}$ không là đồ thị phẳng.



CM : Bốn đỉnh 1, 2, 4, và 5 tạo thành một chu trình và chia mặt phẳng thành 2 vùng R_1 và R_2 . Đỉnh 3 có thể ở R_1 hoặc R_2 . Khi 3 ở R_2 thì ta có 2 vùng R_{21} và R_{22} .



Ví dụ 2 : Đồ thị $K_{3,3}$ không là đồ thị phẳng.



CM : Xét đỉnh 6.

Nếu 6 ở R_1 thì cạnh $(3,6)$ cắt một trong 4 cạnh a, b, c, d .

Nếu 6 ở R_{21} thì cạnh $(2,6)$ cắt một trong 4 cạnh a, d, e, f .

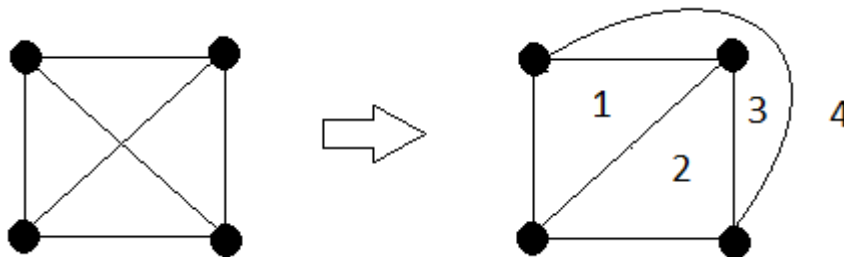
Nếu 6 ở R_{22} thì cạnh $(1,6)$ cắt một trong 4 cạnh c, b, e, f .

Chứng minh tương tự khi 3 nằm ở R_1 . (BT)

7.1.2 Mệnh đề (EULER'S FORMULA)

Cho G là đồ thị phẳng liên thông đơn giản có m cạnh và n đỉnh. Gọi r là số vùng tạo bởi G khi biểu diễn G ở dạng phẳng. Ta có $r = m - n + 2$.

Ví dụ : $n = 4, m = 6, r = 4 = 6 - 4 + 2$



7.1.3 Mệnh đề :

Nếu G là đồ thị phẳng liên thông đơn giản, thì G có một đỉnh có bậc ≤ 5 .

7.1.4 Mệnh đề :

Nếu G là đồ thị phẳng liên thông đơn giản có n đỉnh ,
 $n \geq 3$, m cạnh , thì $m \leq 3n - 6$.

7.1.5 Mệnh đề :

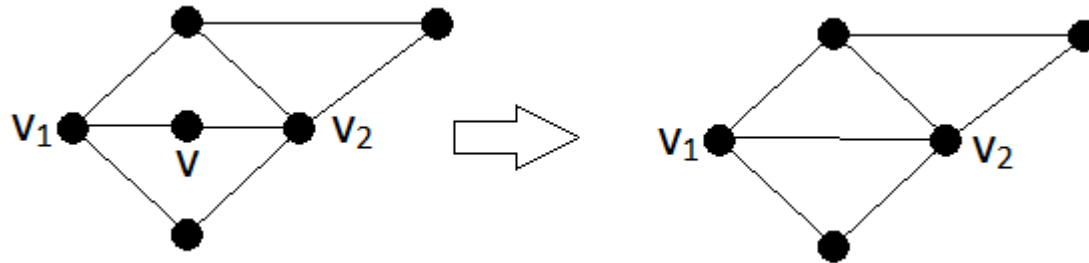
Nếu G là đồ thị phẳng liên thông đơn giản có n đỉnh ,
 $n \geq 3$, m cạnh, và không có chu trình sơ cấp chiều dài
3, thì $m \leq 2n - 4$.

Ví dụ : Dùng mệnh đề **7.1.5** chứng minh $K_{3,3}$ không là đồ thị phẳng.

CM : $K_{3,3}$ là đồ thị liên thông đơn giản có số đỉnh $n = 6$, không có chu trình sơ cấp chiều dài 3, số cạnh $m = 9$, nhưng $2n - 4 = 8$.

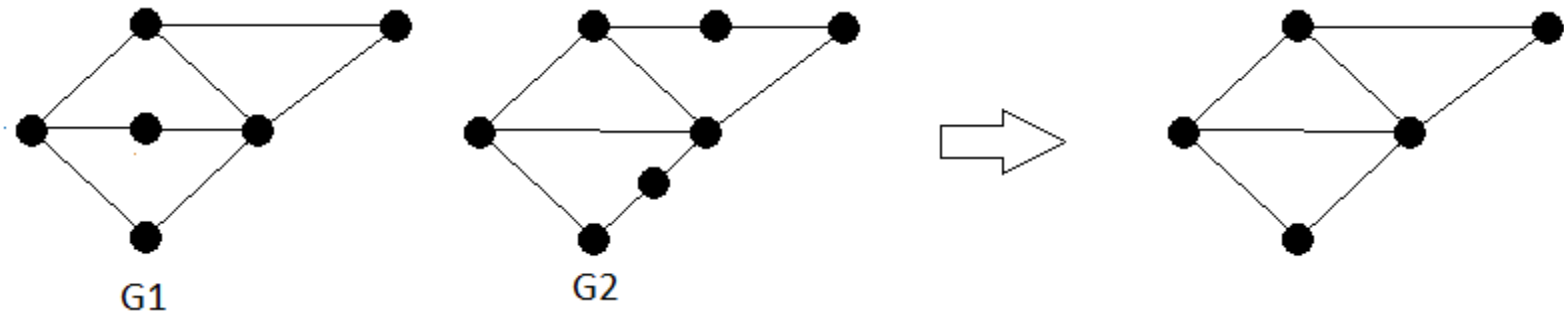
7.1.6 Định nghĩa :

- Nếu đồ thị G có 1 đỉnh v có bậc 2 và các cạnh (v, v_1) và (v, v_2) , với $v_1 \neq v_2$, ta nói rằng cạnh (v, v_1) và (v, v_2) trong cùng 1 dãy (in *series*).
- Một rút gọn dãy (*series reduction*) là xóa đi đỉnh v và thay 2 cạnh (v, v_1) và (v, v_2) bởi cạnh (v_1, v_2) .



7.1.7 Định nghĩa :

Đồ thị G_1 và G_2 là *homeomorphic* G_1 và G_2 có thể được rút gọn thành đồ thị G_1' và G_2' mà G_1' và G_2' là *isomorphic* (đẳng cấu) bởi một chuỗi các rút gọn dãy được định nghĩa ở 7.1.7.

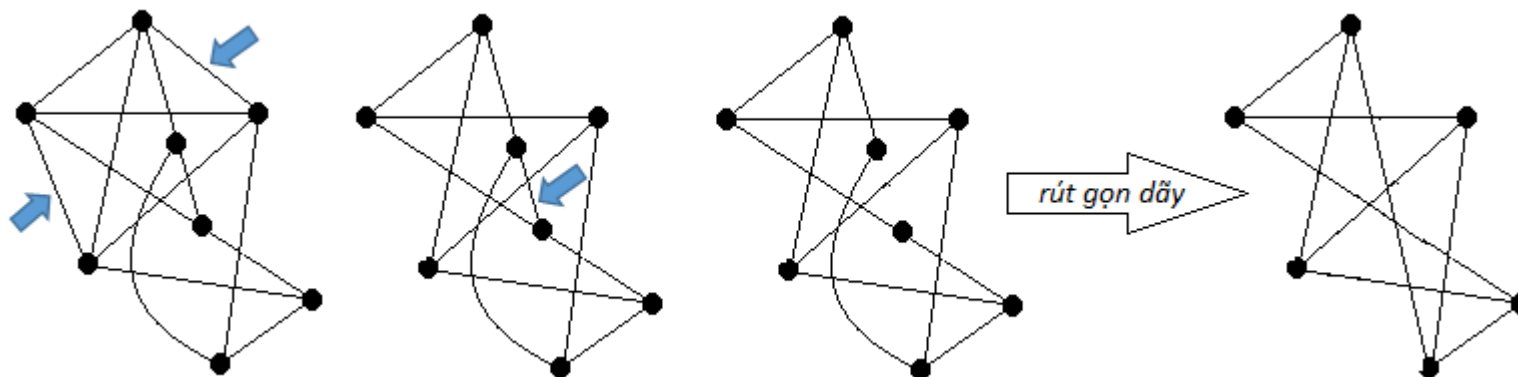


- Đồ thị G luôn được xem là *homeomorphic* với chính nó.

7.1.8 Mệnh đề (Kuratowski's theorem) :

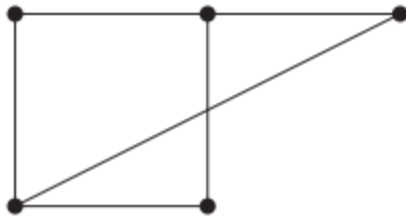
Đồ thị G là phẳng khi và chỉ khi không chứa đồ thị con homeomorphic với $K_{3,3}$ hay K_5 .

Ví dụ :



Bài tập :

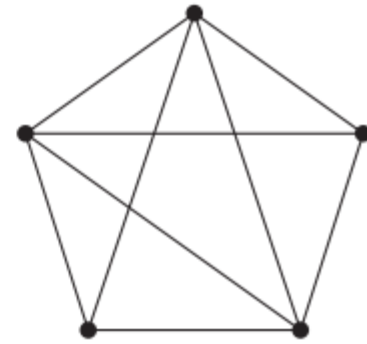
1. Chứng minh các đồ thị sau là phẳng.



G1

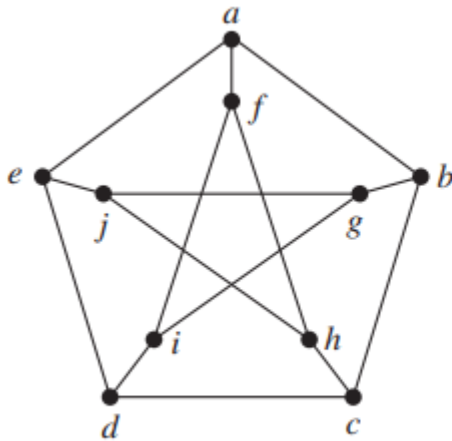


G2

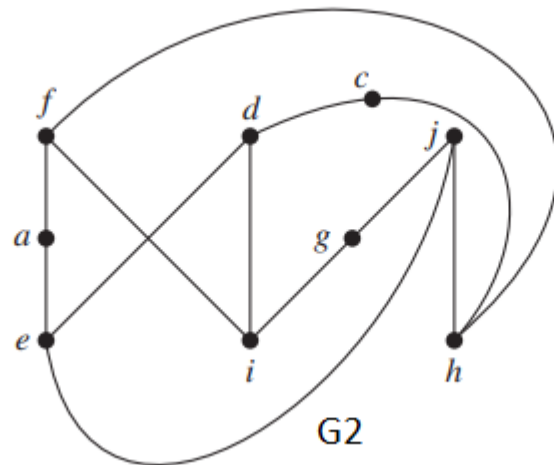


G3

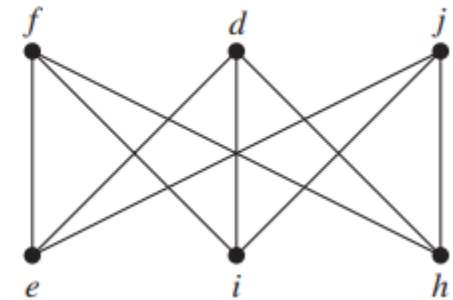
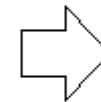
2. Chứng minh 2 đồ thị sau không phẳng.



G1

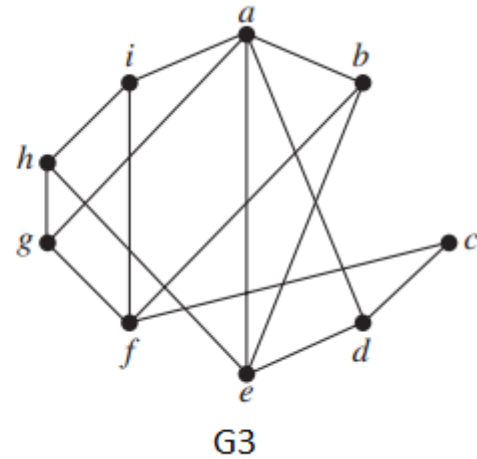
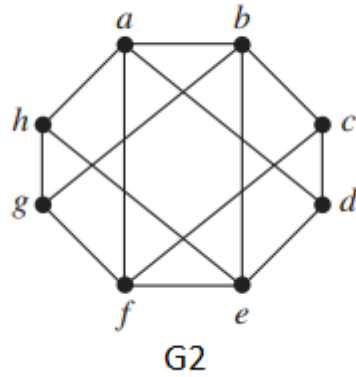
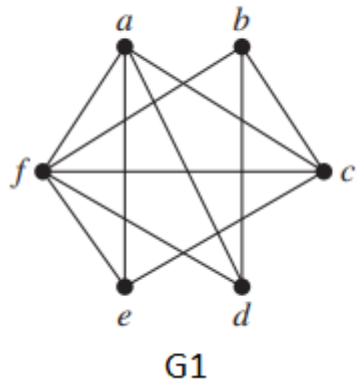


G2



Bài tập :

3. Tìm các đồ thị phẳng phẳng.



7.2 Tô màu đồ thị (Graph Coloring):

7.2.1 Định nghĩa :

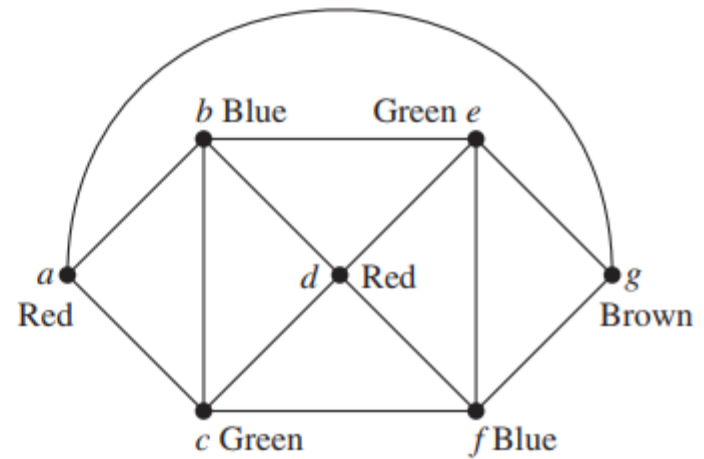
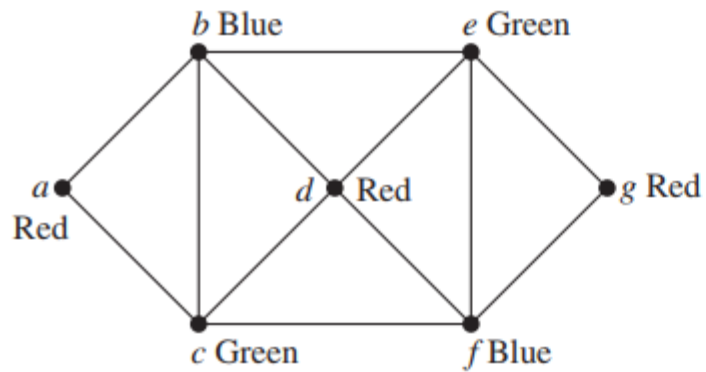
Cho G là đồ thị đơn giản. Tô màu đồ thị G là gán cho mỗi đỉnh của G một màu sao cho hai đỉnh bất kỳ kề nhau thì có màu khác nhau.

7.2.2 Định nghĩa :

Số màu ít nhất dùng tô màu đồ thị G được gọi là *số màu của G* (*chromatic number of G*), ký hiệu là $\chi(G)$ (đọc là *chi*).

7.2.3 Mệnh đề :

Số màu của một đồ thị phẳng ≤ 4 .



7.2.4 Thuật toán tô màu :

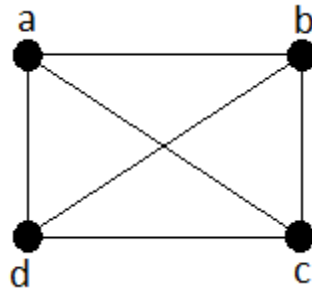
Bước 1 : Tạo một danh sách các đỉnh có bậc giảm dần, $d(v_1) \geq d(v_2) \geq \dots \geq d(v_n)$.

Bước 2 : Gán màu 1 cho v_1 và đỉnh kế tiếp trong danh sách không kề với v_1 , và tiếp tục gán màu 1 cho đỉnh kế tiếp trong danh sách nếu nó chưa được gán màu và không kề với một đỉnh có màu 1.

Gán màu 2 cho đỉnh đầu tiên , v_i , trong danh sách chưa được gán màu và đỉnh kế tiếp trong danh sách không kề với v_i , và tiếp tục gán màu 2 cho đỉnh kế tiếp trong danh sách nếu nó chưa được gán màu và không kề với một đỉnh có màu 2.

Lặp lại cho màu 3, ...

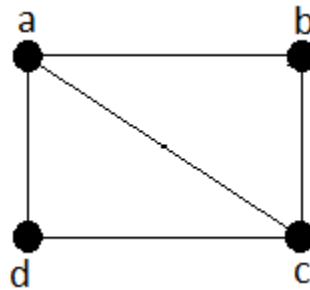
Ví dụ 1 :



Bước 1 : $d(a) \geq d(b) \geq d(c) \geq d(d)$

Bước 2 : Gán màu 1 cho a. Vì các đỉnh kế tiếp trong danh sách đều kề với a nên thực hiện gán màu 2 cho b. Lý luận tương tự đỉnh b được gán màu 3 và đỉnh c được gán màu 4.

Ví dụ 2 :



Bước 1 : $d(a) \geq d(c) \geq d(b) \geq d(d)$

Bước 2 : Gán màu 1 cho a. Vì các đỉnh kề tiếp trong danh sách đều kề với a nên thực hiện gán màu 2 cho c. Vì các đỉnh kề tiếp trong danh sách đều kề với c nên thực hiện gán màu 3 cho b. Đỉnh kết tiếp trong danh sách không kề với b và chưa được tô là d, gán màu 3 cho d.

Bài tập :

